

Computergraphik I

Sichtbarkeitsalgorithmen | Culling



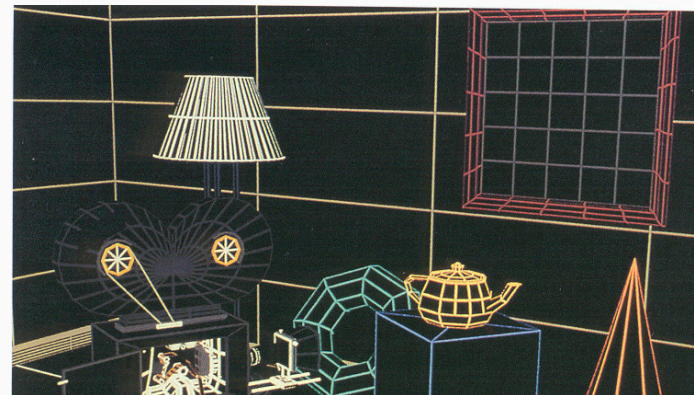
Prof. Dr. Michael Guthe

Sommersemester 2026

AI 5: Graphische Datenverarbeitung
Universität Bayreuth

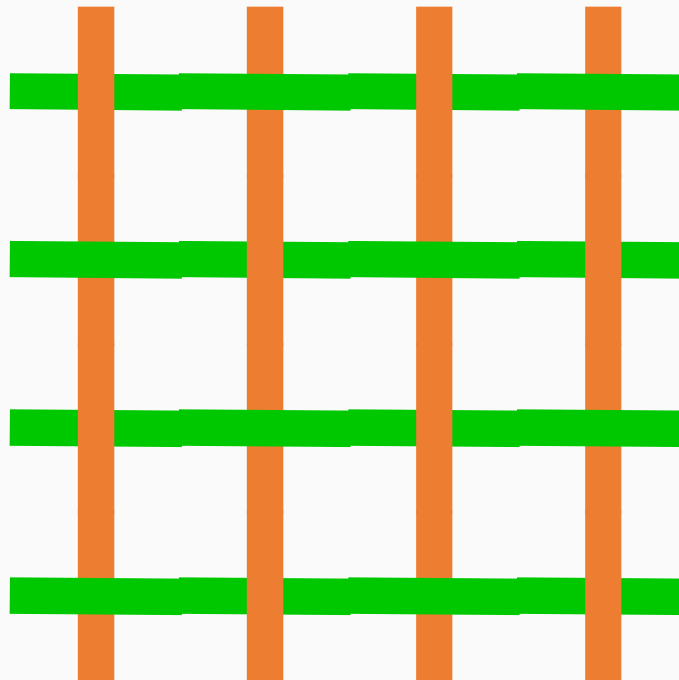
Sichtbarkeitsverfahren (1)

- **Verdeckung** entsteht, wenn mehrere Objekte nach Projektion gleiche Bildschirmkoordinaten haben (**Projektionsäquivalenz**)
 - **Sichtbar** ist der **dem Auge am nächsten** liegende Punkt
 - Ist dieses Objekt **durchsichtig (transparent)**, wird der **dahinterliegende Punkt** auch **sichtbar**, usw.



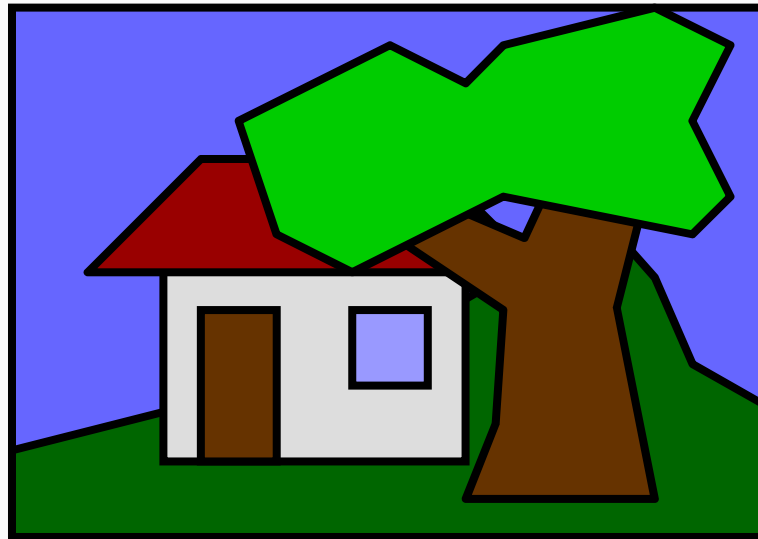
Sichtbarkeitsverfahren (2)

- Bei einer Szene mit n Polygonen, können $O(n^2)$ sichtbare Fragmente entstehen:



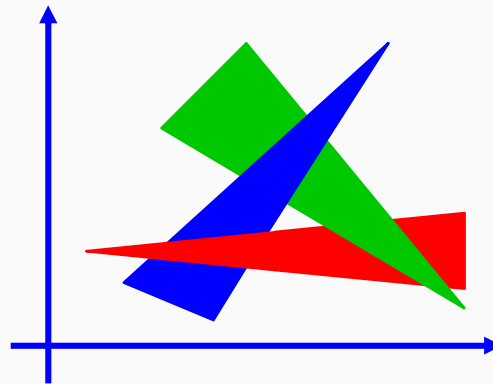
Painter's Algorithmus (1)

- Idee: Zeichne Bild wie ein Maler
 - zuerst den Hintergrund
 - dann Objekte von hinten nach vorne



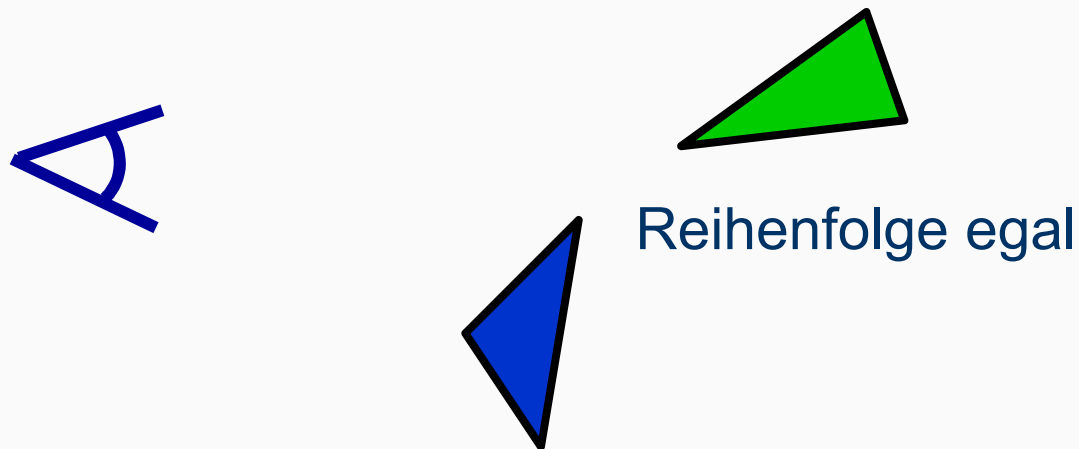
Painter's Algorithm (2)

- Algorithmus:
`sortiere` Dreiecke von hinten nach vorne
`for` jedes Dreieck
 zeichne Dreieck auf Bildschirm
- Probleme:
 - Sortieren hat $O(n \log n)$ Laufzeit.
 - Oft existiert eine Sortierung gar nicht (Zyklen)



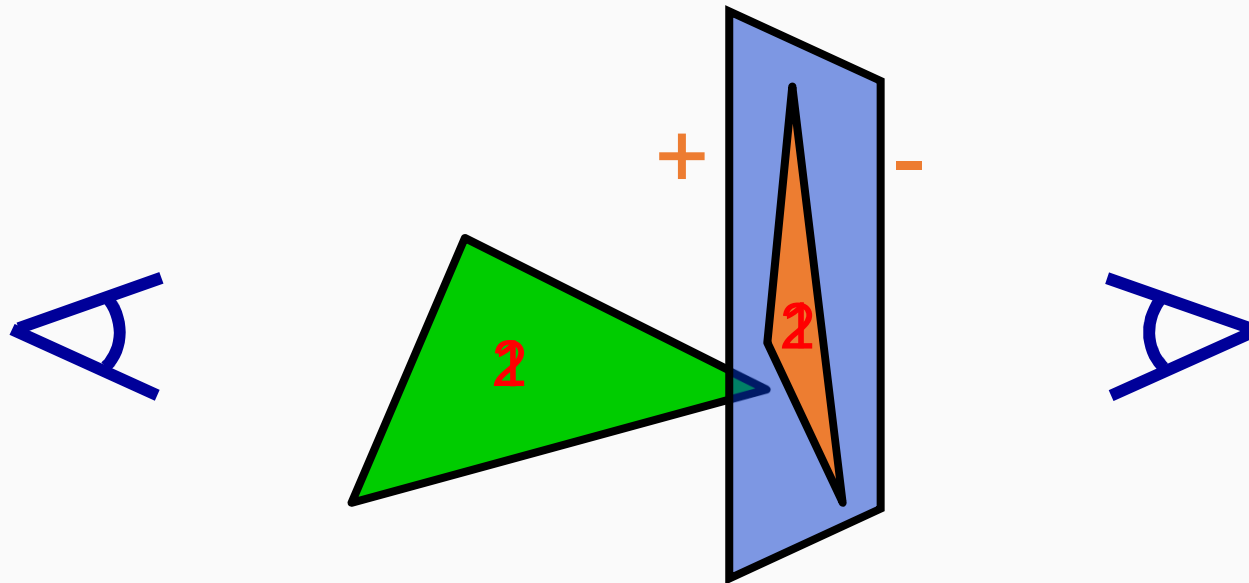
Painter's Algorithmus (3)

- Lösungsansatz (Problem 1):
 - Exakte Tiefensortierung nicht nötig
 - Vorverarbeitung der Dreiecke (BSP-Baum)
 - Durchlaufen des Baumes bei Blickpunktänderung



Painter's Algorithmus (4)

- Grundprinzip des BSP-Baums:
 - Jedes Dreieck definiert eine Ebene
 - Unterteilt den Raum in zwei Halbräume
 - Reihenfolge abhängig vom Betrachterhalbraum

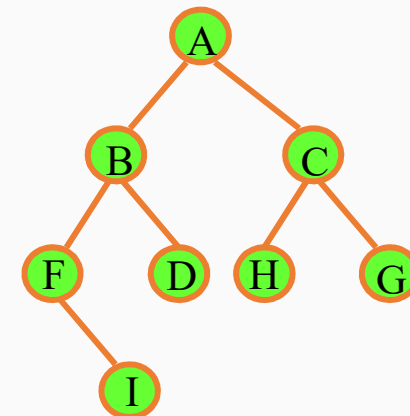
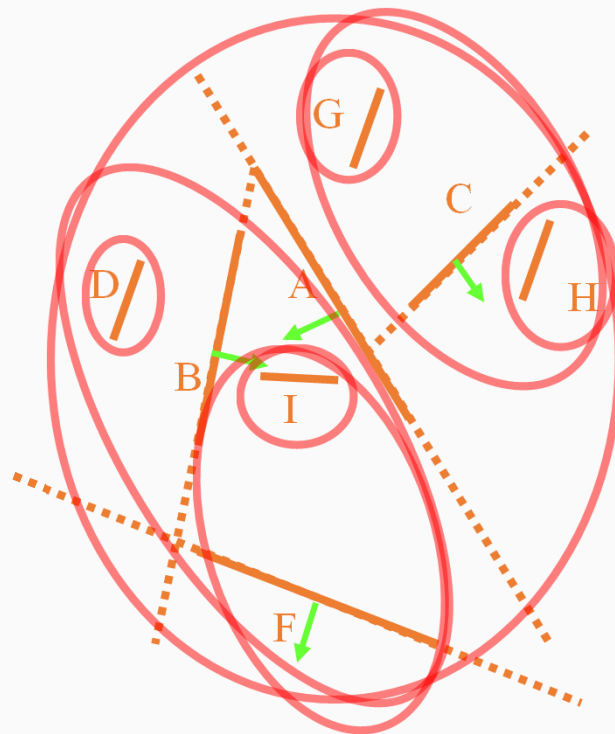


Painter's Algorithmus (5)

- Ein BSP-Baum über der Menge S ist ein Binärbaum T mit folgenden Eigenschaften:
 - Falls $\text{card}(S)=1$, ist der Knoten v ein Blatt und speichert S
 - Sonst speichert v die Hyperebene H mit der Ebenengleichung $E(\mathbf{p})=0$ und alle vollständig in H enthaltenen Elemente:
 $S_0 := \{\mathbf{s} \in S : E(\mathbf{s})=0\}$
 - Kindknoten v_+ und v_- enthalten alle im positiven bzw. negativen Halbraum liegenden Elemente:
 $S_+ := \{\mathbf{s} \in S : E(\mathbf{s})>0\}$ bzw. $S_- := \{\mathbf{s} \in S : E(\mathbf{s})<0\}$

Painter's Algorithmus (6)

- Erzeugung des BSP-Baums:
 - Rekursive Unterteilung des Raumes bis nur noch ein Element pro Zelle übrig



Painter's Algorithmus (7)

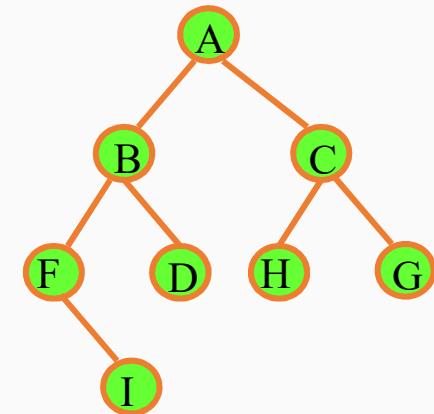
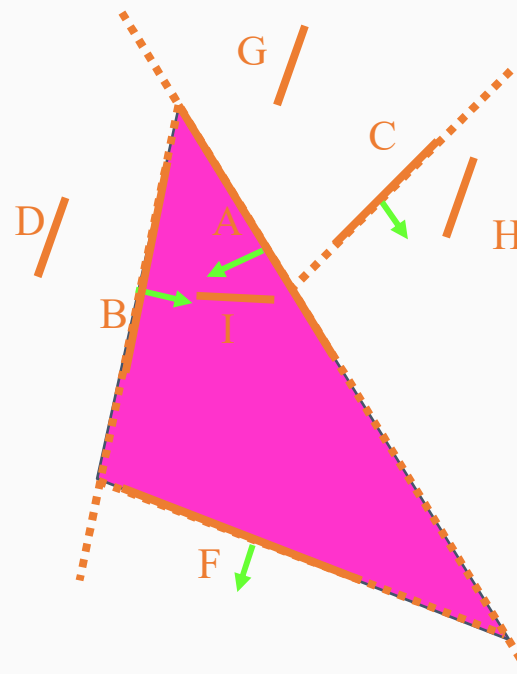
- Bemerkung:

Jeder Knoten n korrespondiert zu einer konvexen Menge R_n , nämlich dem Durchschnitt aller Halbräume H_μ^* , wobei μ Vorfahre von n ist und $*$ = +, falls n im linken

Teilbaum von μ
bzw. $*$ = -, falls n

im rechten

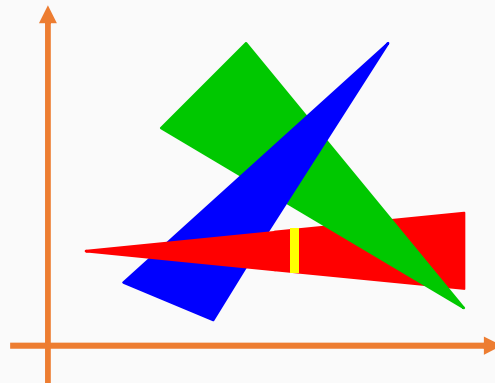
Teilbaum
von μ liegt



$$R_I = H_A^+ \cap H_B^+ \cap H_F^-$$

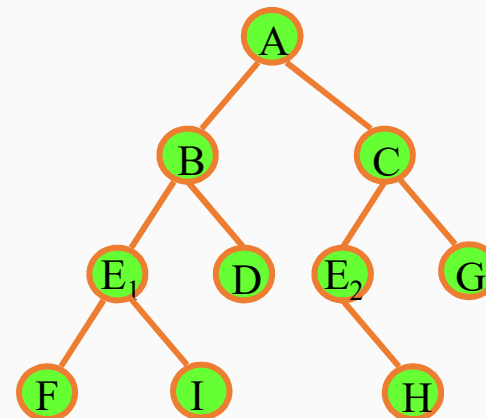
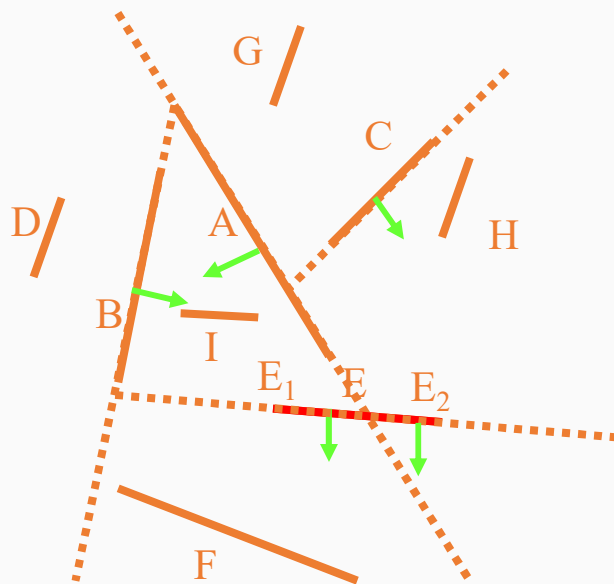
Painter's Algorithmus (8)

- Lösungsansatz (Problem 2):
 - Zerschneiden einiger Dreiecke, so daß immer eine Sortierung möglich ist
 - Kann in die Baumerzeugung integriert werden



Painter's Algorithmus (9)

- Lösungsansatz (Problem 2):
 - Integration in Baumerzeugung:
 - Zerschneide Dreiecke mit aktueller Ebene



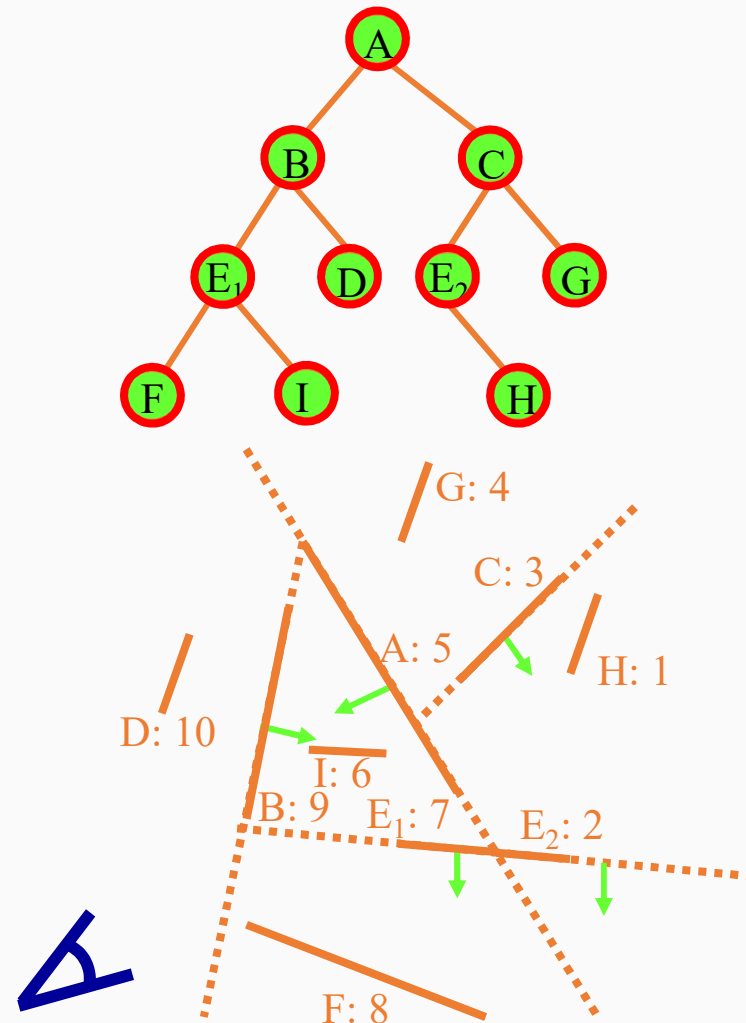
Painter's Algorithmus (10)

Painter(v, a) :

```
if v ist Blatt  
    zeichne(v.S)
```

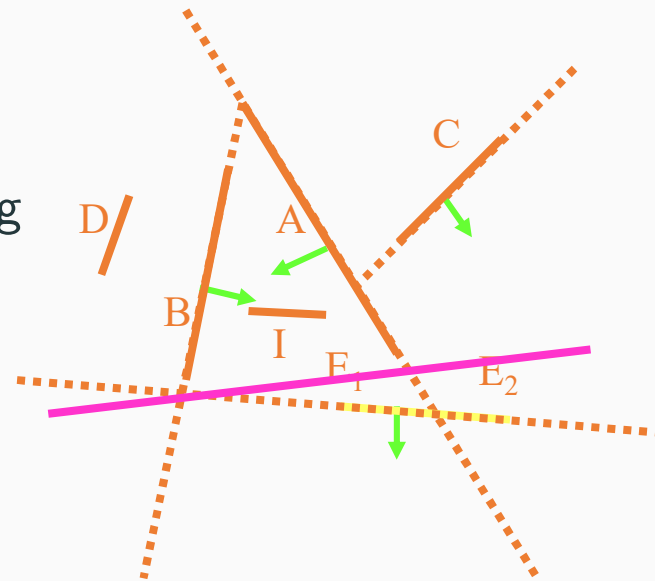
```
elseif E(a) < 0  
    → Painter(v.v+, a)  
    → zeichne(v.S0)  
    → Painter(v.v-, a)
```

```
else  
    → Painter(v.v-, a)  
    → zeichne(v.S0)  
    → Painter(v.v+, a)
```



Konstruktion von 2D BSP-Bäumen

- Gegeben: $S = \{s_1, \dots, s_n\}$ Menge von Liniensegmenten, die sich nicht schneiden
- Gesucht: (möglichst kleiner) BSP-Baum
- Lösung: **Randomisierter Algorithmus** erzeugt BSP-Baum der Größe $O(n \log n)$ in $O(n^2 \log n)$ Zeit
 - Idee:
 1. Erzeuge zufällige Permutation
 2. Wähle immer das erste Liniensegment zur Unterteilung
- Optimierungsmöglichkeit: "Free Splits"

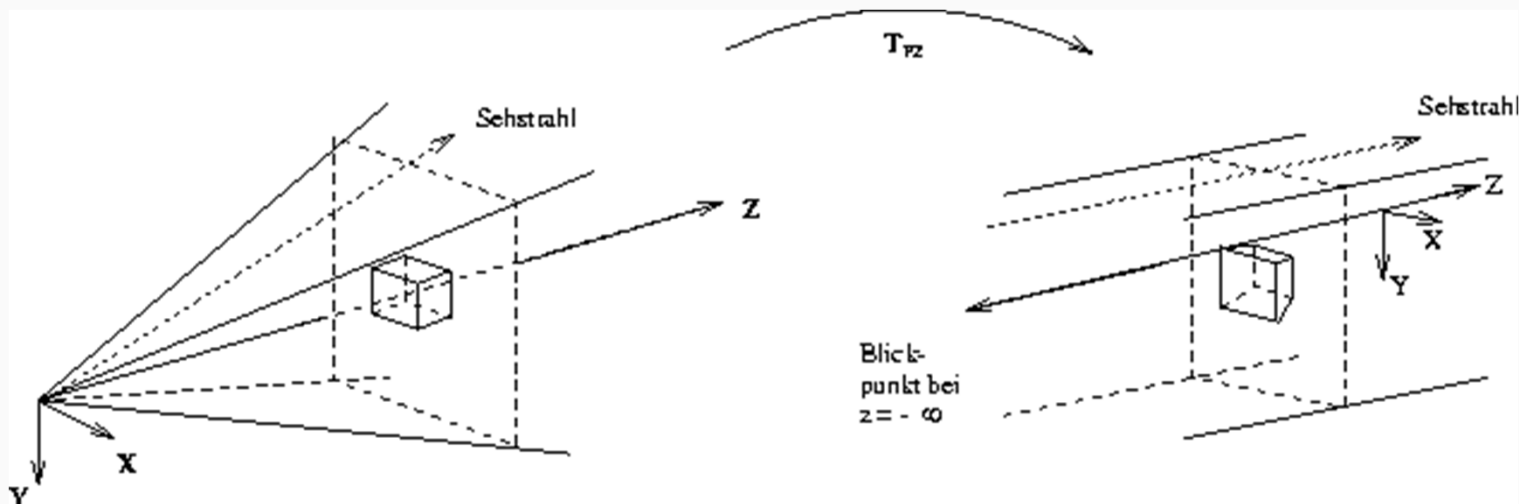


Konstruktion von 3D BSP-Bäumen

- Gegeben: $S = \{t_1, \dots, t_n\}$ Menge von Dreiecken, die sich nicht schneiden
- Gesucht: (möglichst kleiner) BSP-Baum
- Lösung: **Randomisierter Algorithmus** erzeugt BSP-Baum der Größe $O(n^2)$ in $O(n^3)$ Zeit
- Bemerkungen:
 - $O(n^2)$ ist eine untere Schranke, d.h. es gibt Datensätze, für die jeder BSP-Baum die Größe $O(n^2)$ hat
 - Für orthogonale Rechtecke können BSP-Bäume der Größe $O(n^{1,5})$ erzeugt werden.

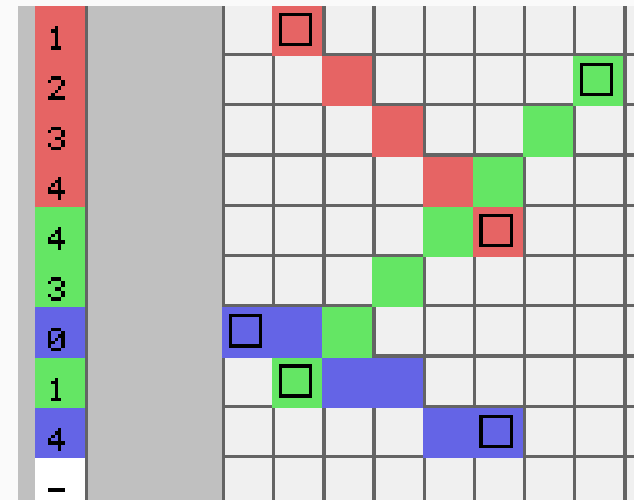
Tiefenvergleich im Bildraum

- Feststellung der Projektionsäquivalenz wesentlich einfacher nach perspektivischer Transformation
 - Da Tiefenrelation unverändert, reduziert sich die Bestimmung des sichtbaren Punktes auf einen Tiefenvergleich (z-Werte)
 - Im folgenden: Linkssystem, Augpunkt nach Transformation: $-\infty$



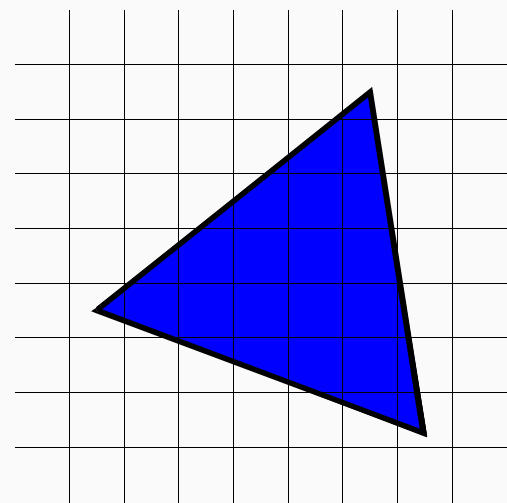
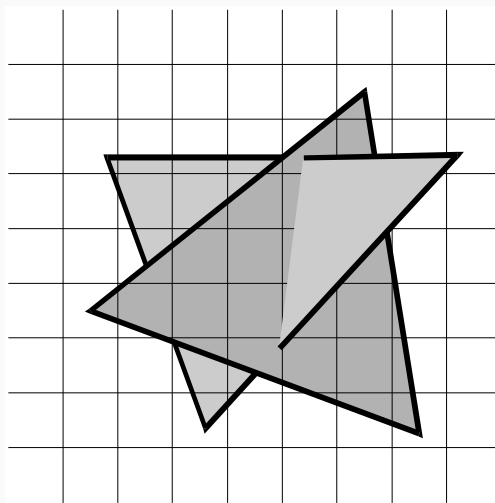
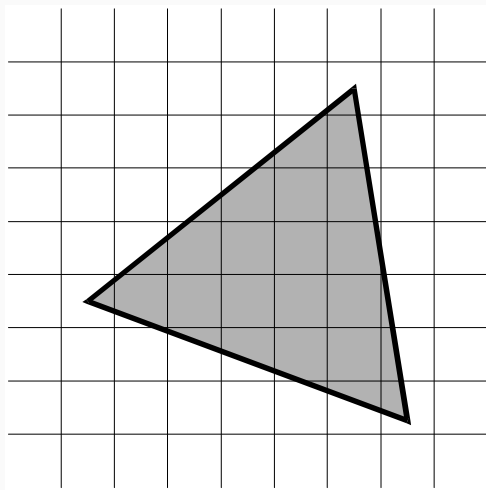
Punktorientierte Algorithmen

- Einzelne Punkte werden auf Sichtbarkeit untersucht
 - Einfachste Art der Sichtbarkeitsberechnung
 - Benötigen aber größte Anzahl von Tests
 - Liefern die Sichtbarkeit explizit
 - Werden für Rastergeräte eingesetzt, weil dann Test- und Ausgabeelement übereinstimmen
 - Bekanntester Vertreter ist der z-Buffer-Algorithmus



Rasteralgorithmen

- In der Rastereinheit werden Dreiecke und Linien in Rasterpunkte (Pixel) zerlegt



- Attribute (Farbe, etc.) werden dabei mit baryzentrischen Koordinaten interpoliert

Wdh.: Pineda Algorithmus (1)

- Berechnung der Baryzentrischen Koordinaten:

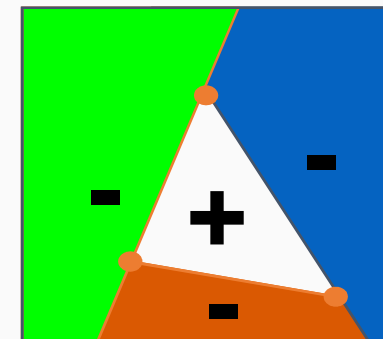
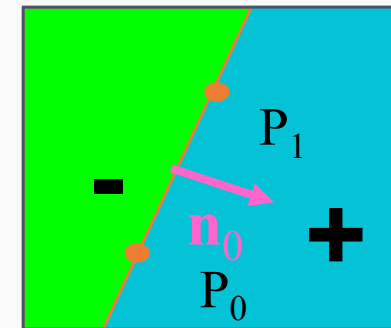
$$P_i = (x_i, y_i)^t$$

$$\mathbf{n}_i = (\Delta y_i, -\Delta x_i)^t = (y_{i+1} - y_i, x_i - x_{i+1})^t$$

$$P = (x, y)^t$$

$$\lambda_i(P) = \frac{A(\Delta(P, P_{i+1}, P_{i+2}))}{A(\Delta(P_0, P_1, P_2))}$$

$$= \frac{\mathbf{n}_{i+1} \cdot P - \mathbf{n}_{i+1} \cdot P_{i+1}}{\mathbf{n}_1 \cdot P_0 - \mathbf{n}_1 \cdot P_1}$$



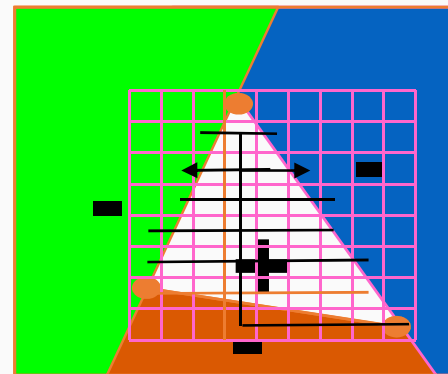
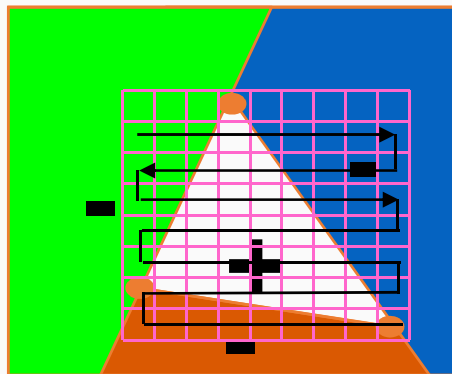
Wdh.: Pineda Algorithmus (2)

- $\lambda_i(P)$ kann inkrementell berechnet werden

$$\lambda_i(x+1, y) = \lambda_i(x, y) + \frac{\Delta y_{i+1}}{\mathbf{n}_1 \cdot \mathbf{P}_0 - \mathbf{n}_1 \cdot \mathbf{P}_1}$$

$$\lambda_i(x, y+1) = \lambda_i(x, y) - \frac{\Delta x_{i+1}}{\mathbf{n}_1 \cdot \mathbf{P}_0 - \mathbf{n}_1 \cdot \mathbf{P}_1}$$

- Traversierung:



z-Buffer Algorithmus (1)

- Wurde bereits 1974 von W. Straßer vorgeschlagen, aber aus Kostengründen damals nicht realisiert
- Prinzip vergleichbar mit **Selection Sort**:
 - Für jeden Bildpunkt wird der Farbwert des vordersten Primitives behalten
 - Neben dem **Farbwert** wird für jeden Bildpunkt auch ein **z-Wert** gespeichert

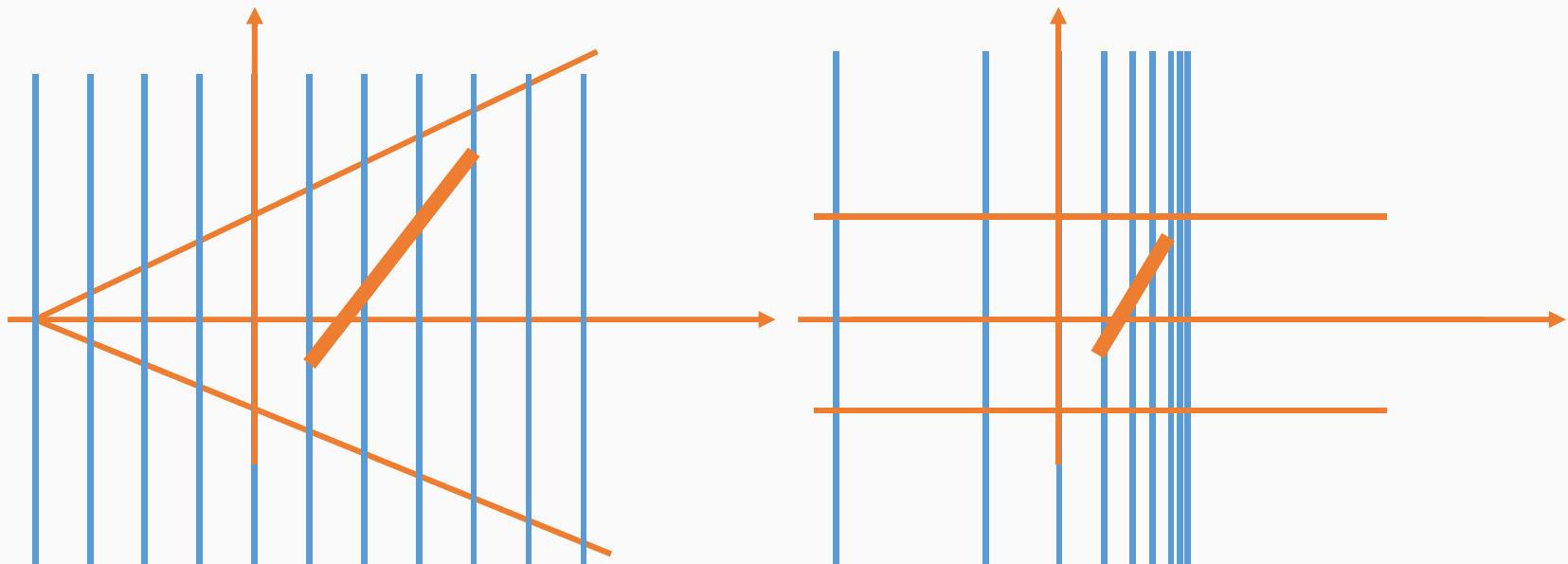
z-Buffer Algorithmus (2)

- Ablauf:
 - **Bildspeicher** wird mit **Hintergrundfarbe** und **z-Speicher** mit **maximalem z-Wert** initialisiert
 - Rastern aller Primitive nacheinander
 - **Besondere Reihenfolge** ist **nicht erforderlich**
 - Für jeden Punkt (x,y) und für jedes Primitiv durchläuft der Algorithmus folgende Schritte:

z-Buffer Algorithmus (3)

1. Berechne $z(x,y)$

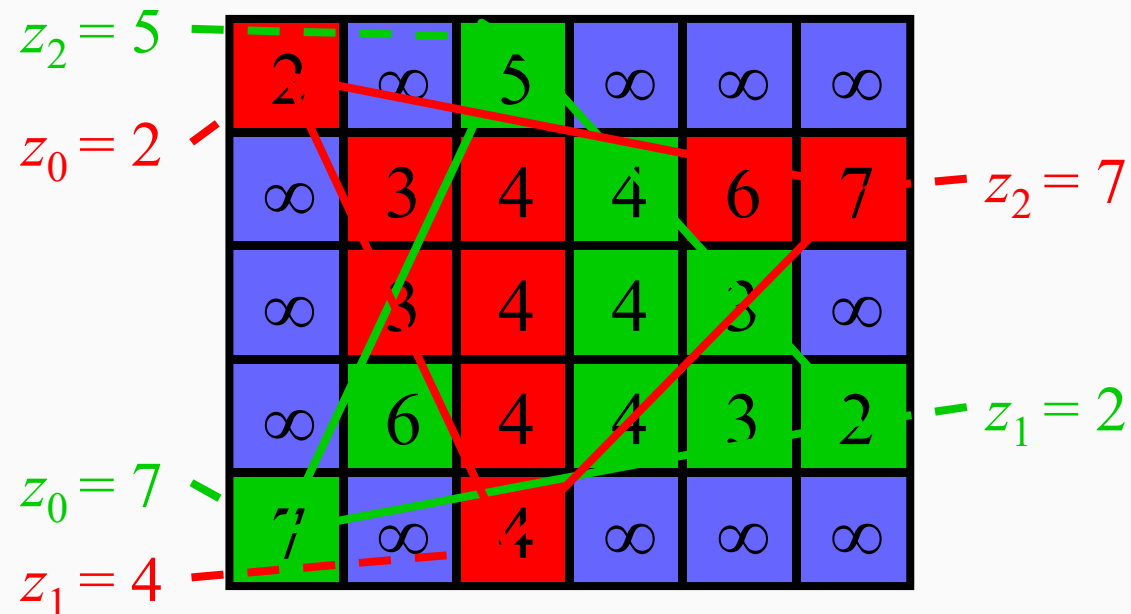
- Auch auf perspektivisch transformiertem Dreieck durch lineare Interpolation



z-Buffer Algorithmus (4)

2. Ist $z(x,y)$ kleiner als der unter (x,y) bereits gespeicherte z-Wert:

- Schreibe $z(x,y)$ in den z-Speicher und Farbwert an Stelle (x,y) in den Bildspeicher



z-Buffer Algorithmus (5)

```
void display()
{
    glPushAttrib(GL_DEPTH_BUFFER_BIT);
    glClear(GL_COLOR_BUFFER_BIT |
           GL_DEPTH_BUFFER_BIT);
    glEnable(GL_DEPTH_TEST);

    // Zeichnen
    ...

    glPopAttrib(GL_DEPTH_BUFFER_BIT);
    glFlush();
}
```

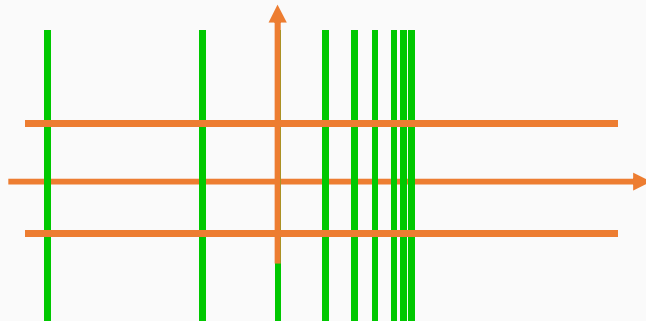
- Löschen des z-Buffers mit „glClear“ und dem Flag „GL_DEPTH_BUFFER_BIT“
- Aktivieren des z-Buffer Test mit „glEnable“ und dem Flag „GL_DEPTH_TEST“
- Sichern und wiederherstellen des OpenGL Zustands mit „glPushAttrib“ und „glPopAttrib“
 - „GL_DEPTH_TEST“ gehört zur Attributgruppe „GL_DEPTH_BUFFER“

z-Buffer Algorithmus (6)

- Vorteile:
 - Jede Szene kann unabhängig von der Komplexität behandelt werden
 - Nachträgliches Einfügen von Objekten in fertige Szene möglich
 - Besonders für die Kombination mit Kameraaufnahmen interessant
 - Spezielle Objekte (z.B. 3D-Cursor) können mit korrekter Verdeckung dargestellt werden
 - Leicht in Hardware zu realisieren

z-Buffer Algorithmus (7)

- Nachteile:
 - Pro Bildpunkt nur ein Primitiv gespeichert
 - Einige fortgeschrittene Techniken (Transparenz, u.a.) benötigen aber alle Primitive
 - Genauigkeit des z-Buffers ist beschränkt
 - Integerwerte, um Speicherplatz zu sparen
 - Uniforme Verteilung nach der Projektion entspricht nichtuniformer vor der Projektion

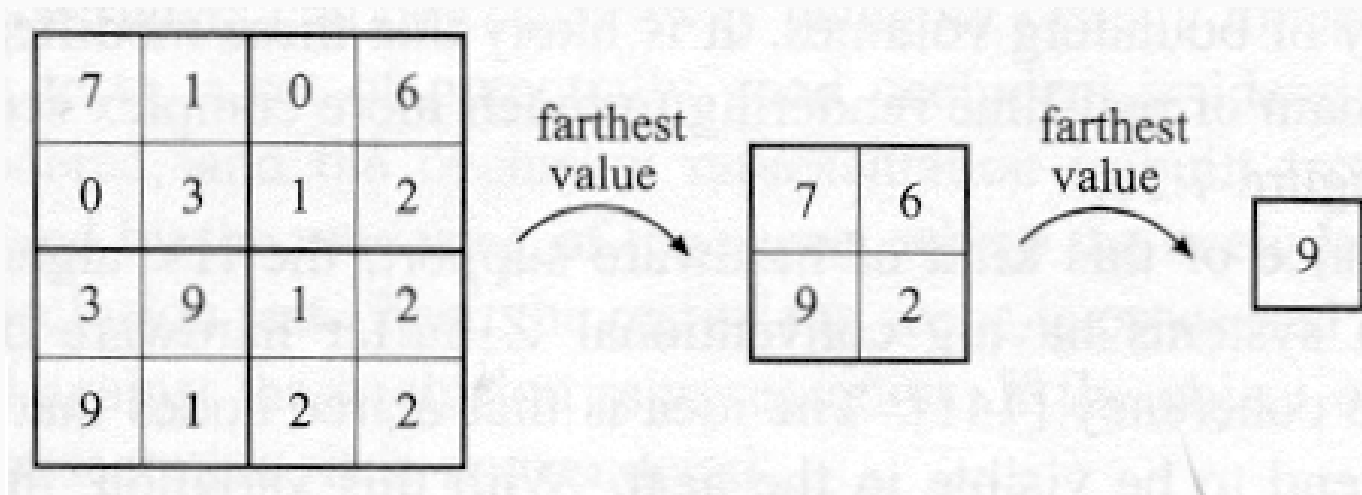


$$z_i = \frac{2fn}{n + f - (f - n)\frac{2i-b}{b}}$$

$$z_b - z_{b-1} \approx \frac{f}{n}(n - f)\frac{1}{b}$$

Hierarchischer Z-Buffer (HZB) (1)

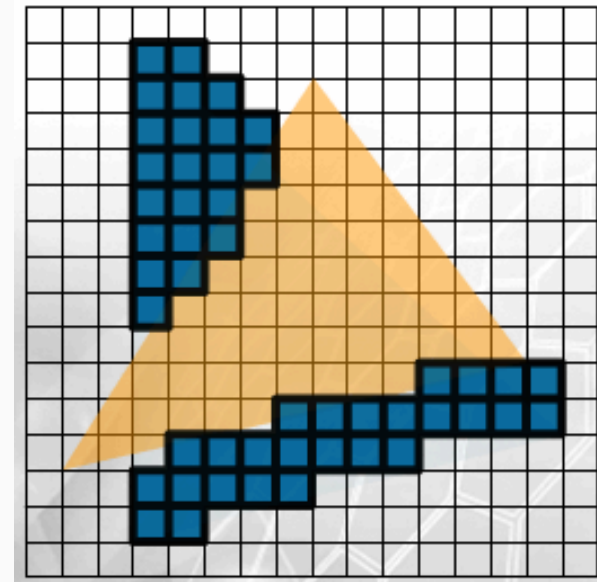
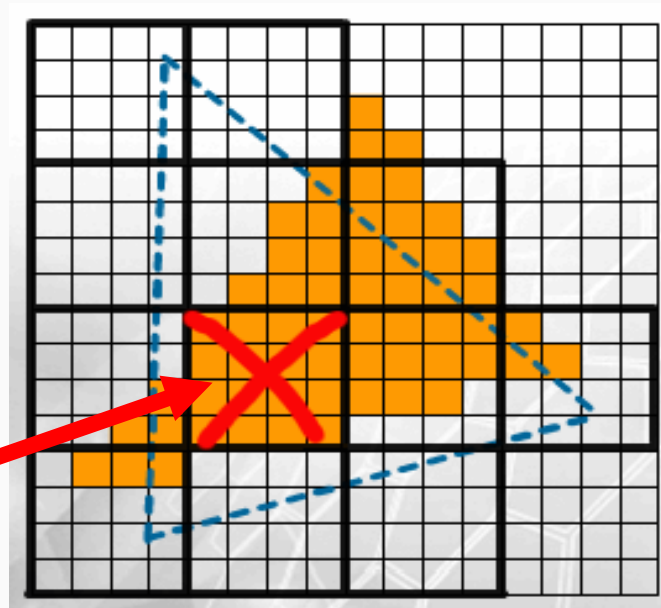
- Z-Pyramide
 - Höchste Auflösung: einfacher z-Buffer
 - Weitere Ebenen durch Zusammenfassen von jeweils 4 Pixeln
 - z-Wert auf max. z-Wert setzen



Hierarchischer Z-Buffer (HZB) (2)

- Beispiel:
 - Oranges Dreieck bereits gezeichnet
 - Blaues Dreieck soll dahinter gezeichnet werden

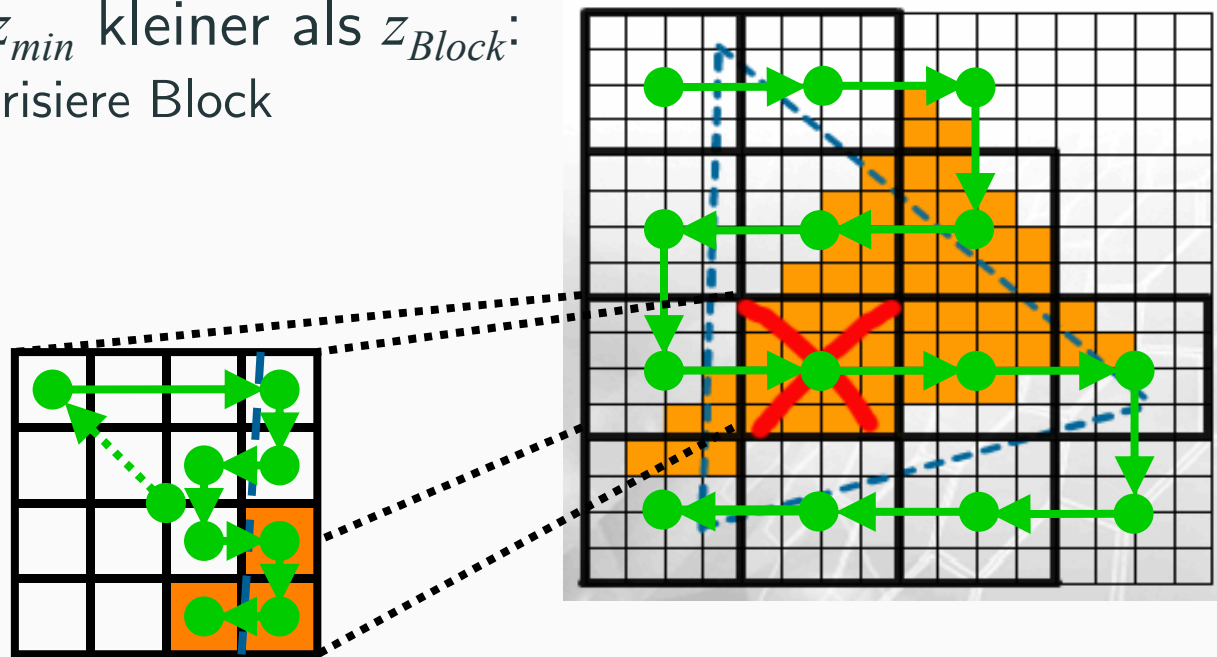
vollständig verdeckt
→ verwurfe gesamten
Block



ATI Radeon (HyperZ), nVidia GeForce (Z-Occlusion-Culling)

Hierarchischer Z-Buffer (HZB) (3)

- Integration in die Rasterisierung:
 - Dreieck auf grober Auflösung Rasterisieren (Pineda Algorithmus)
 - Wenn mindestens ein Pixel belegt ist und z_{min} kleiner als z_{Block} :
 - Rasterisiere Block

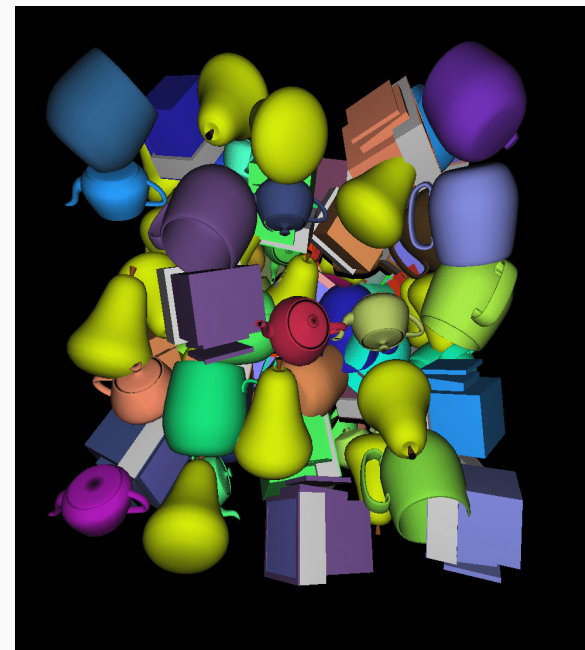
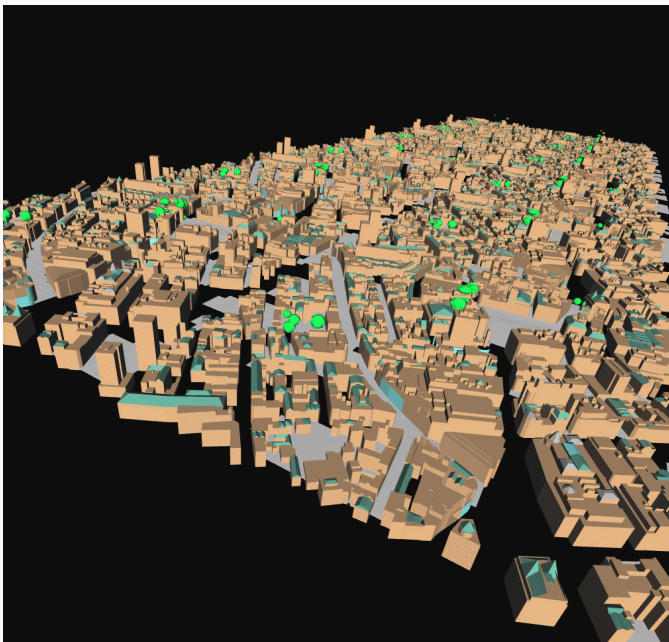


Culling (1)

- Alle bisher betrachteten Algorithmen arbeiten auf der gesamten Menge A aller Primitive
 - Ihr Aufwand ist daher immer mindestens proportional zu $|A|$
- Unproblematisch, wenn die Anzahl der sichtbaren Primitive $|S|$ in etwa $|A|$ ist
 - z.B., wenn die Anzahl der Primitive im Vergleich zur Pixelanzahl klein ist

Culling (2)

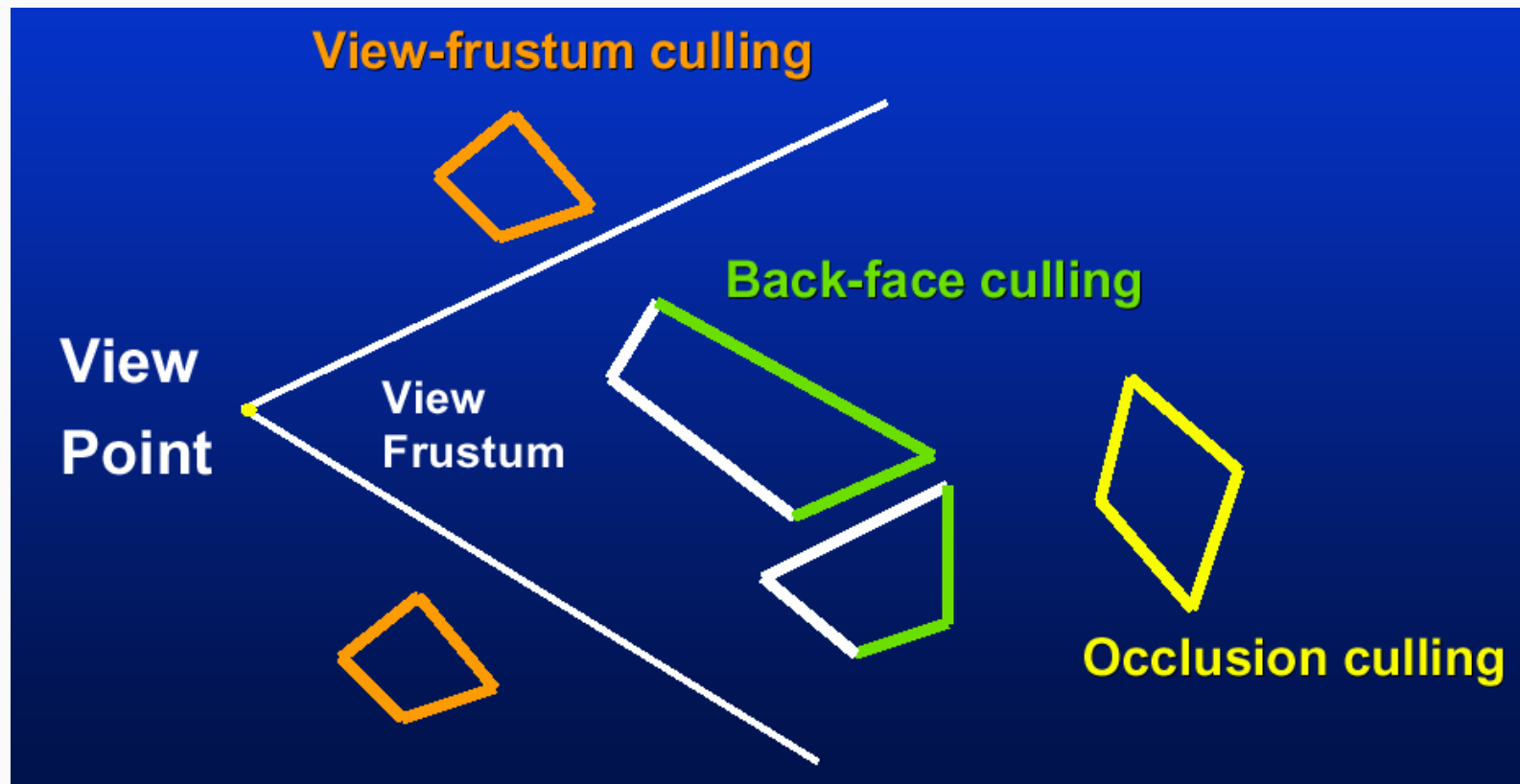
- Aber: für komplexe Szenen ist die Anzahl der sichtbaren Primitive in der Regel wesentlich kleiner als die Anzahl der Primitive ($|S| \ll |A|$)!



Culling (3)

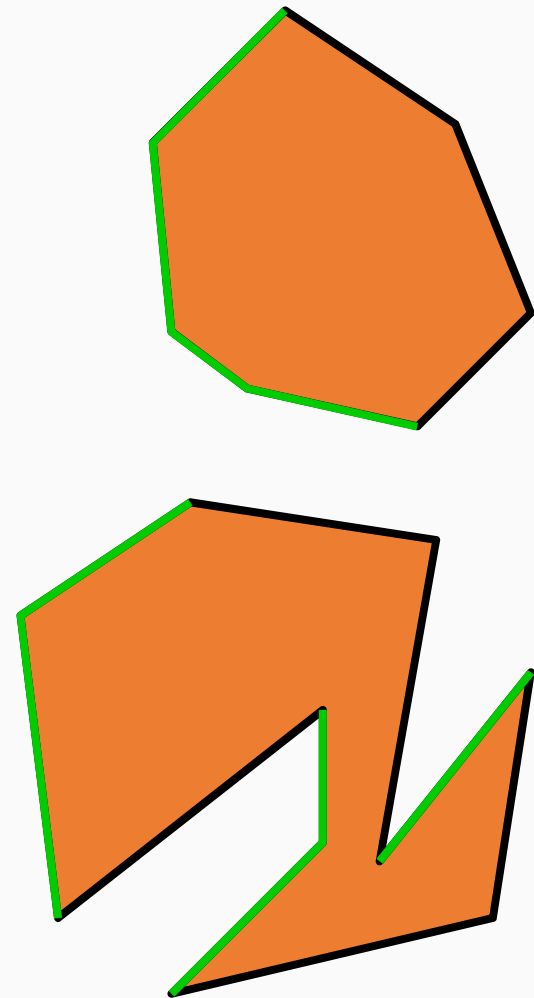
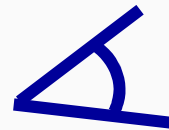
- Für $|S| \ll |A|$ sind die bisherigen Algorithmen zu ineffizient
- Culling Algorithmen versuchen, Menge der nicht sichtbaren Primitive $C=A-S$ (oder Teilmenge davon) zu bestimmen
 - Diese müssen bei der Sichtbarkeitsberechnung nicht mehr berücksichtigt werden

Culling (4)



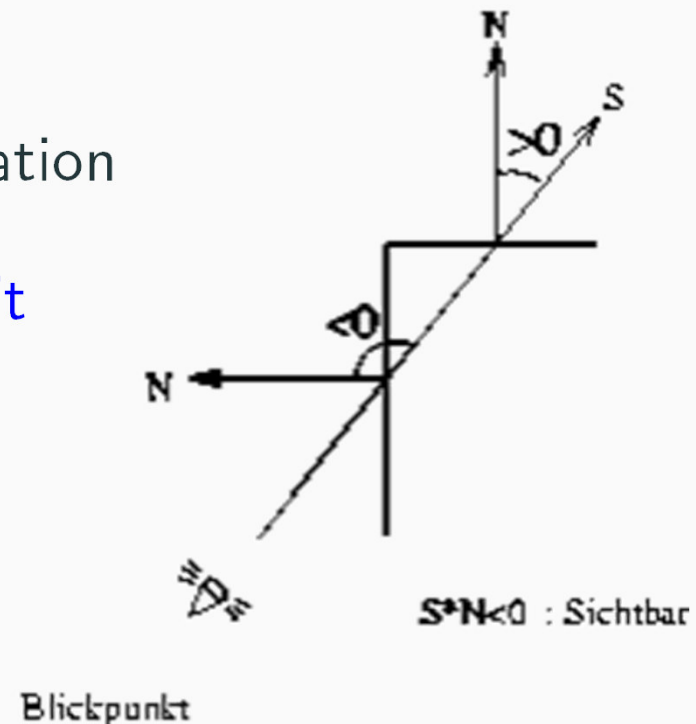
Objektrückseiten – Back-face Culling (1)

- Beobachtungen:
 - Bei einem konvexen Objekt muss nur die Rückseite entfernt werden
 - Bei geschlossenen Objekten sind die Rückseiten nie sichtbar



Objektrückseiten – Back-face Culling (2)

- Der **Normalenvektor** von **Rückseiten** hat im Koordinaten-system des kanonischen Sichtvolumens eine **positive z-Komponente**
 - Können auch vor der perspektivischen Transformation aus dem **Vorzeichen** des **Skalarprodukts Sehstrahl mit Normale** ermittelt werden

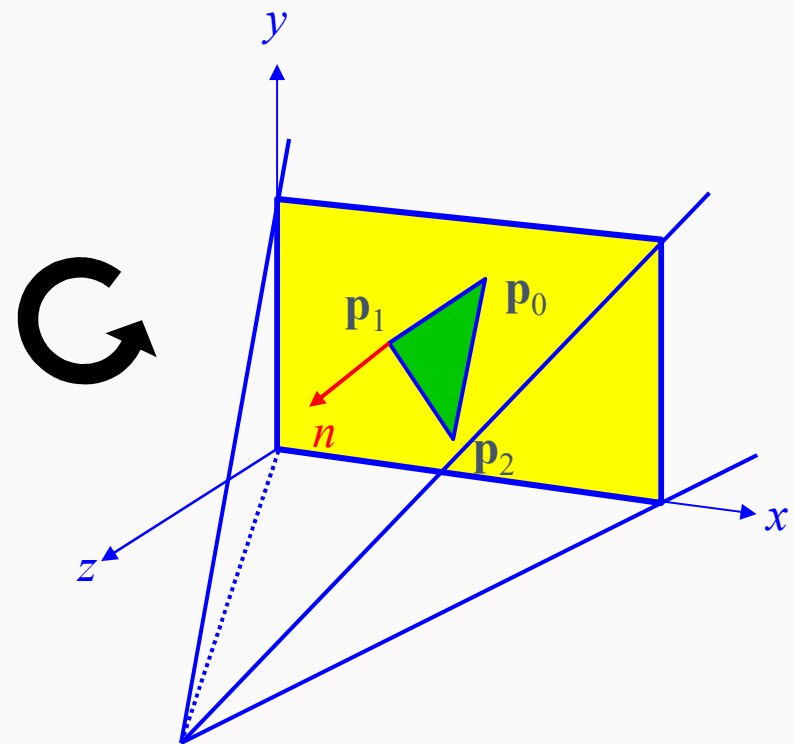


Objektrückseiten – Back-face Culling (3)

- Teste jedes Dreieck (konsistent orientiert) in der Bildebene (xy-Ebene):

$$\begin{aligned} n &= (p_1 - p_0) \times (p_2 - p_0) \\ &= (0, 0, a)^t, a \in \mathbb{R} \end{aligned}$$

- Bei ccw-Orientierung ist das Dreieck für $a > 0$ dem Betrachter zugewandt



Objektrückseiten – Back-face Culling (4)

```
void display()
{
    glPushAttrib(
        GL_DEPTH_BUFFER_BIT |
        GL_POLYGON_BIT);
    glClear(GL_COLOR_BUFFER_BIT |
           GL_DEPTH_BUFFER_BIT);
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_CULL_FACE);

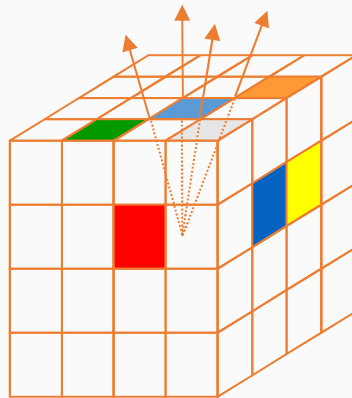
    ... // Zeichnen

    glPopAttrib(
        GL_DEPTH_BUFFER_BIT |
        GL_POLYGON_BIT);
    glFlush();
}
```

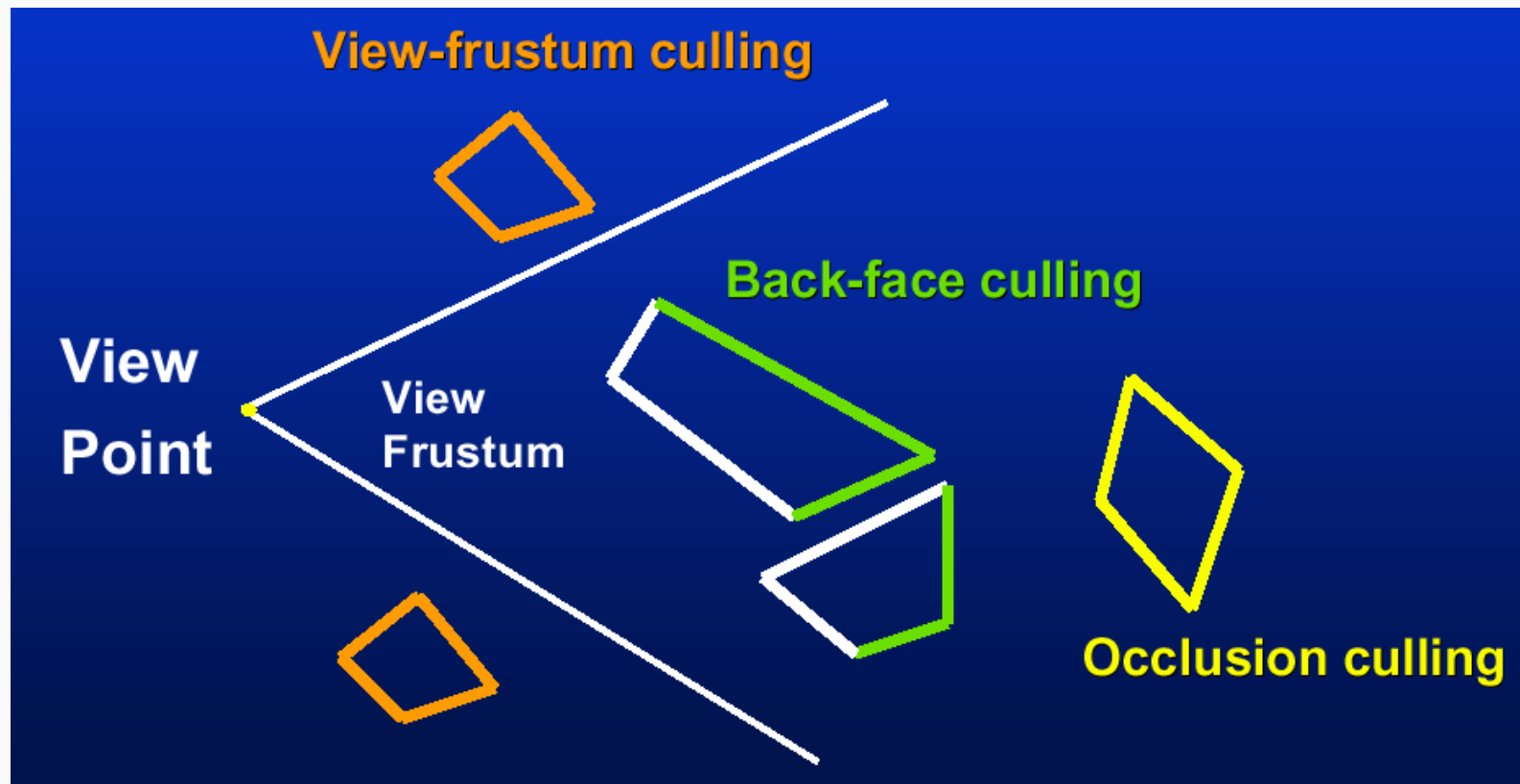
- Aktivieren des Back-face Culling mit „glEnable“ und dem Flag „GL_CULL_FACE“
- Sichern und wiederherstellen des OpenGL Zustands mit „glPushAttrib“ und „glPopAttrib“
 - „GL_CULL_FACE“ gehört zur Attributgruppe „GL_POLYGON“

Objektrückseiten – Back-face Culling (5)

- Optimierung durch **Clusterbildung**:
 - Würfeloberfläche wird gleichmäßig unterteilt
 - Jedem Quadrat wird eine schiefe Pyramide von Normalen zugeordnet
 - In einem Vorverarbeitungsschritt wird jeder Pyramide eine Dreiecksliste anhand der Normalen zugeordnet (Clusterbildung)
 - Culling wird **clusterweise** durchgeführt

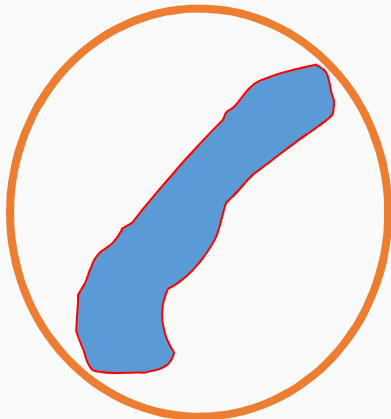


View-Frustum Culling (1)

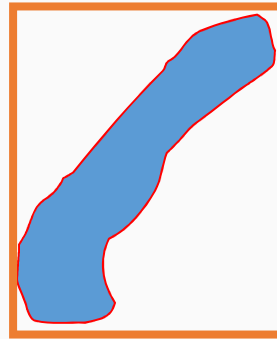


View-Frustum Culling (2)

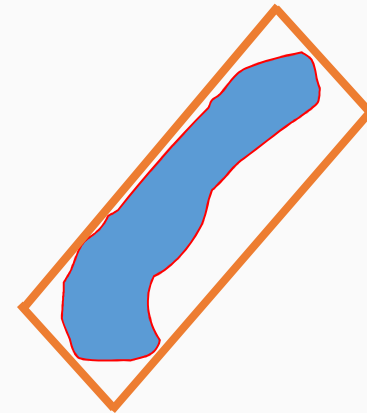
- Test, ob Menge von Primitiven außerhalb des Sichtfrustums
 - Test mit jeder der 6 Ebenen
 - Lassen sich aus den 8 Eckpunkten (in Weltkoordinaten) aufstellen
- Schnelle Tests mit einfachen Hüllkörpern:



Kugel



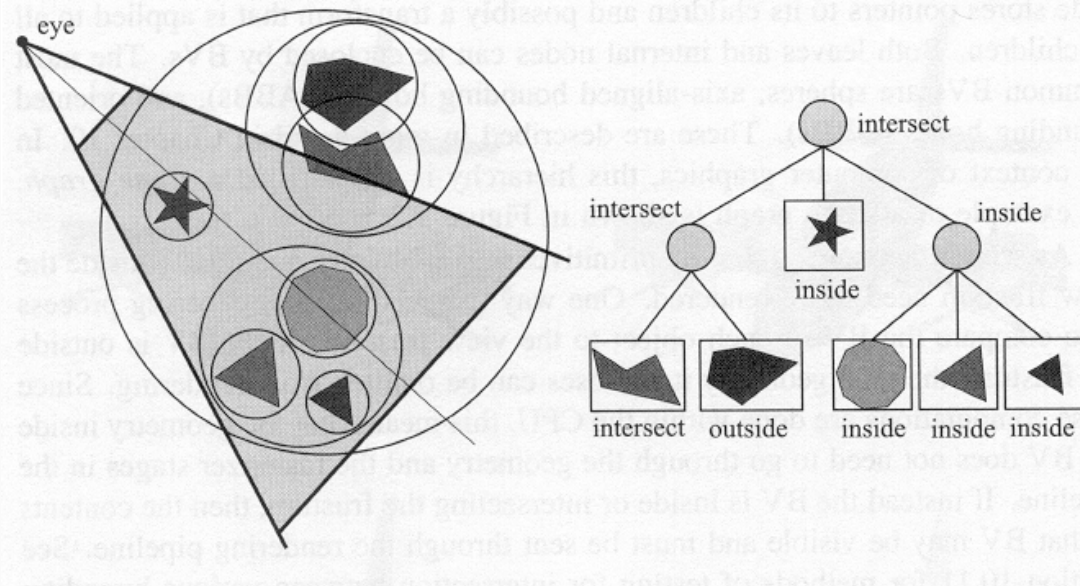
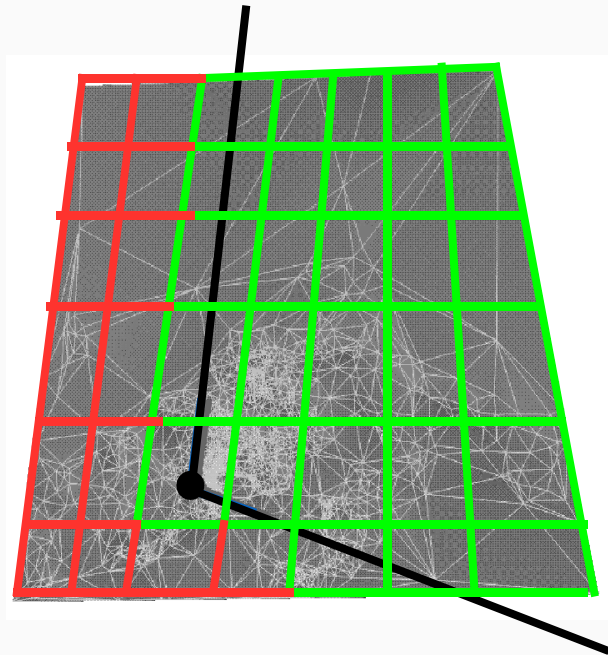
achsenparallele
Bounding Box (AABB)



orientierte BB (OBB)

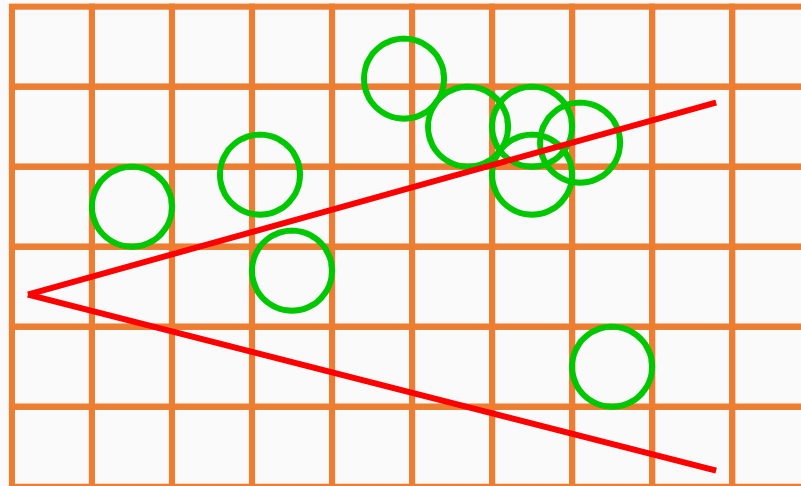
View-Frustum Culling (3)

- Meist entweder einfache Raumunterteilung
 - z.B. achsenparallele Gitter oder Octrees
- Oder Hüllkörperhierarchien



View-Frustum Culling (4)

- Achsenparallele Gitter (Grids)
 - Einfach
 - Objekt ist in mehreren Zellen enthalten
 - Kann sich der Geometrie nicht anpassen

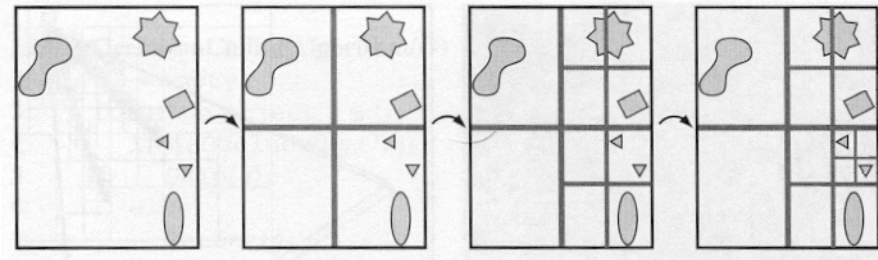


View-Frustum Culling (5)

- Octree

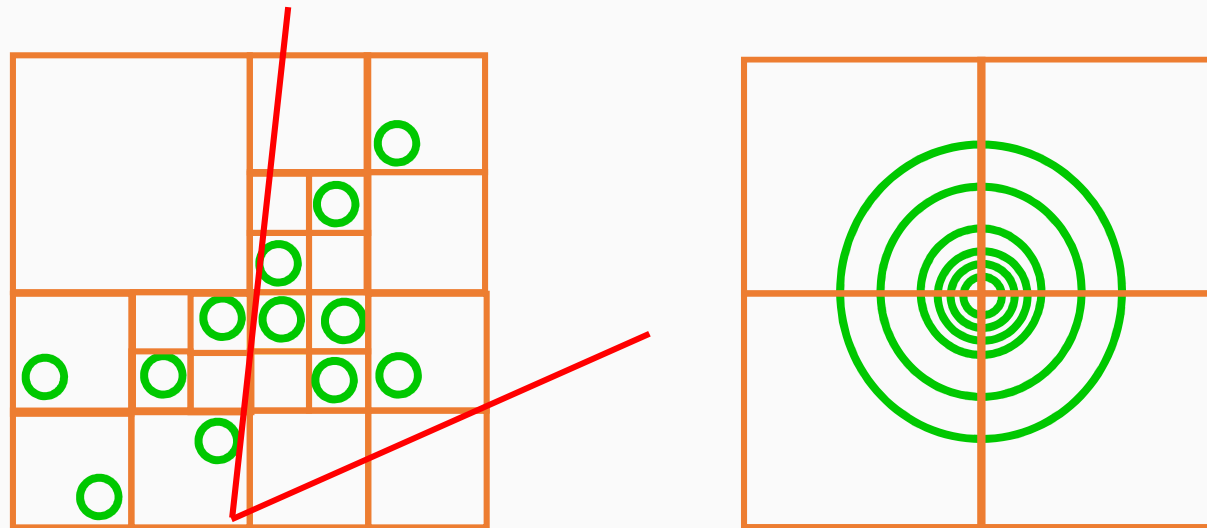
```
subdivide(root, scene.objects)
```

```
subdivide(node, object_list)
    if (object_list.size < threshold)
        node.list = object_list
    else
        for (each object in object_list)
            if (object.intersect(node.separating_planes))
                object_list.remove(object)
                node.list.add(object)
            else
                for (i=0; i<8; ++i)
                    if (object.intersect(node.child[i]))
                        child_list[i].add(object)
        for (i=0; i<8; ++i)
            subdivide(node.child[i], child_list[i])
```



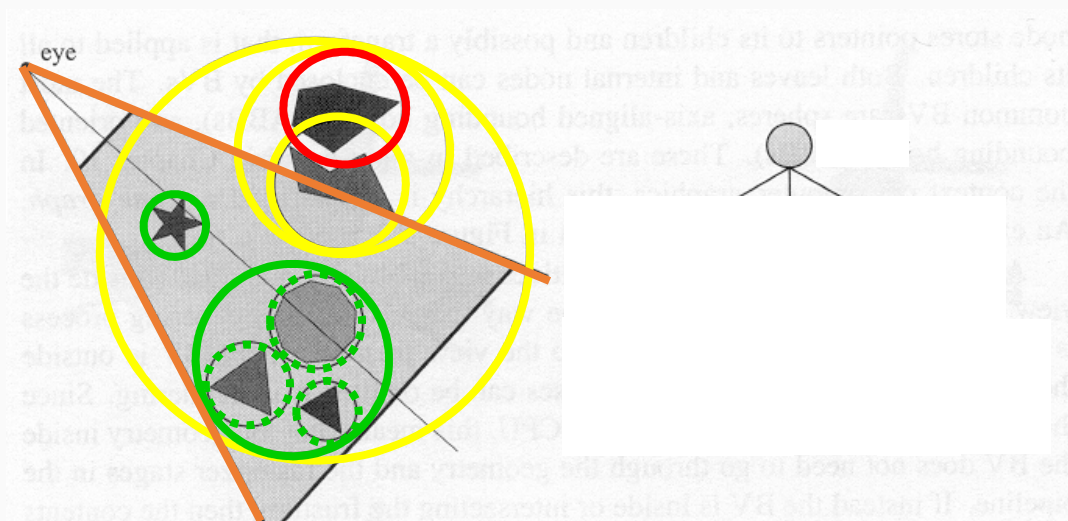
View-Frustum Culling (6)

- Octrees
 - Passen sich der tatsächlichen Geometrie gut an
 - Mehr Speicherbedarf als Grid
 - Problem mit Objekten, die Mittelpunkt einer Zelle enthalten (keine logarithmische Tiefe)



View-Frustum Culling (7)

- Beginne mit der Wurzel des Graphen
- Teste jeden Knoten auf Schnitt mit der Sichtpyramide
 - Falls Schnitt, wird der Inhalt gezeichnet und die Traversierung rekursiv fortgesetzt
 - Falls kein Schnitt, wird die Bearbeitung des Knotens eingestellt



View-Frustum Culling (8)

- Beschleunigungstechniken:
 - Vermeide unnötige Tests
 - Kindknoten müssen nur mit geschnittenen Ebenen getestet werden
 - Benutze Bitcodes bei Traversierung zum Speichern der Ebenen, die noch getestet werden müssen
 - Abbrechen, wenn Hüllkörper komplett außerhalb bezüglich einer Ebene

View-Frustum Culling (9)

- Nützliche OpenGL Methoden:
 - Umrechnen von Pixelkoordinaten in Objektraumkoordinaten

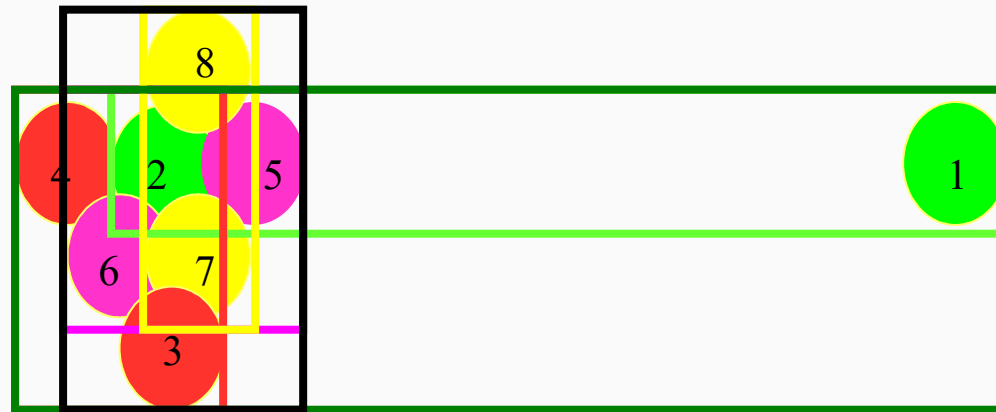
```
double *objPos = new double[3];
gluUnProject(winx, winy, winz,
             model, proj, view,
             objPos );
```
 - Abfragen der Transformationsmatrizen

```
double *model = new double[16];
double *proj = new double[16];
glGetDoublev(GL_MODELVIEW_MATRIX, model);
glGetDoublev(GL_PROJECTION_MATRIX, proj);
```
 - Abfragen des Zeichenbereichs (x_0, y_0, w, h)

```
int *view = new int[4];
glGetIntegerv(GL_VIEWPORT, view);
```

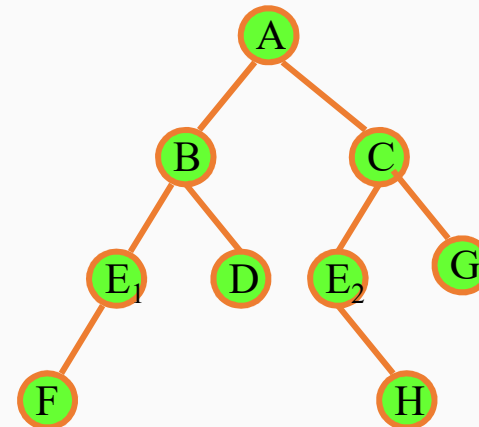
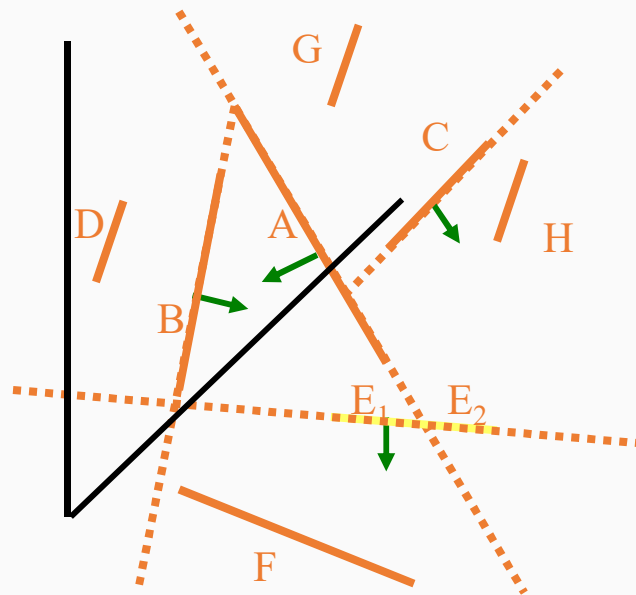

Hüllkörperhierarchien (1)

- Vorgegebene Reihenfolge liefert nicht automatisch gute Hierarchie:



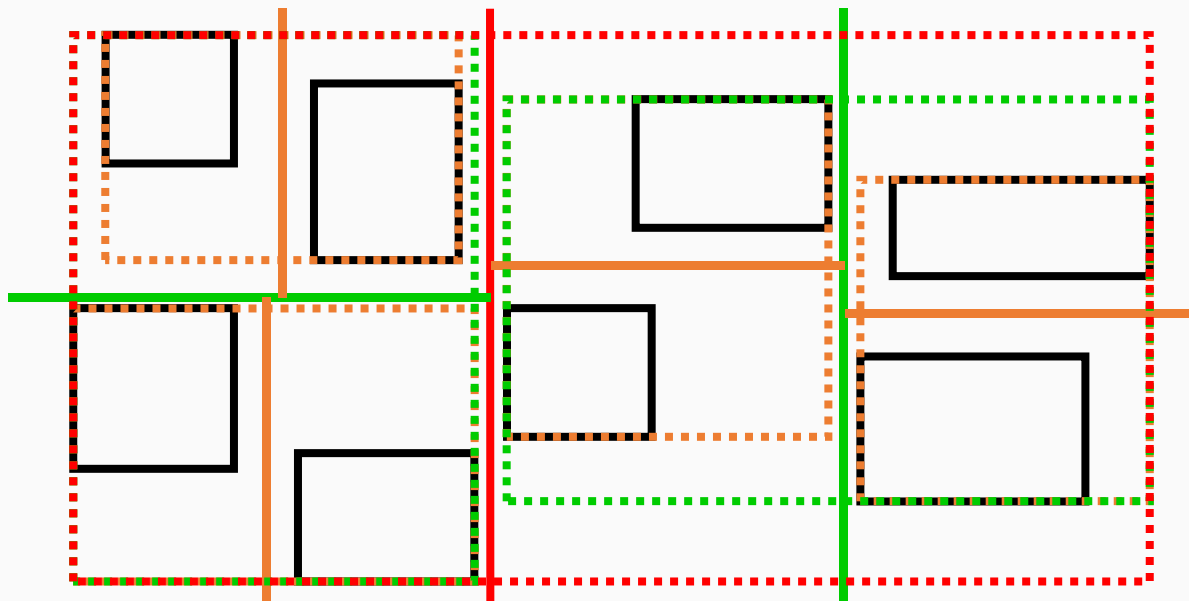
Hüllkörperhierarchien (2)

- BSP-Bäume
 - Konstruktion wie bei Painter's Algorithmus
 - Schnittest von Halbraum mit Sichtfrustum
 - Traversierungsreihenfolge meist unwichtig



Hüllkörperhierarchien (3)

- kD-Bäume (für AABB Hierarchien)
 - Binärbaum mit achseparalleler Unterteilungsebene
 - Bei Schnitt von Objekt mit Ebene, entweder in beiden Kindknoten enthalten, oder nach Mittelpunkt zugeordnet



Hüllkörperhierarchien (4)

- Welche Eigenschaften hat eine gute Hierarchie?
 - Logarithmische Tiefe
 - Gesamtoberfläche der Hüllkörper minimal
 - Minimiert Wahrscheinlichkeit, dass Hüllkörper das Sichtfrustum schneidet und Objekt außerhalb liegt
- Aber: Berechnungsaufwand für optimale Hierarchie ist $O(e^n)$!

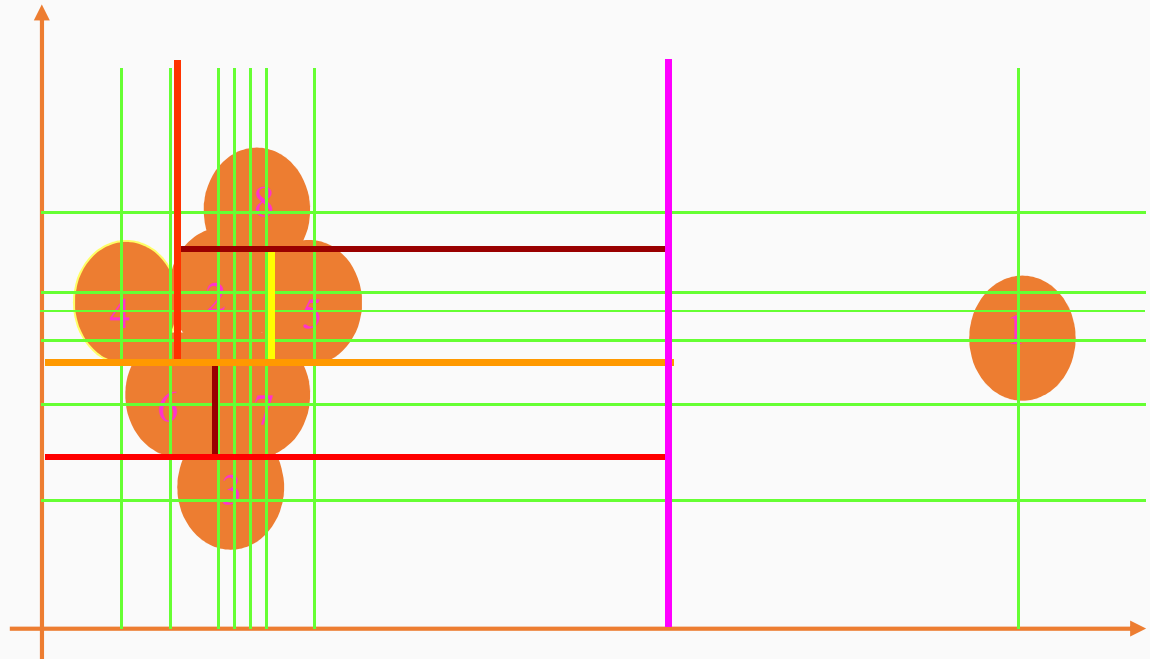
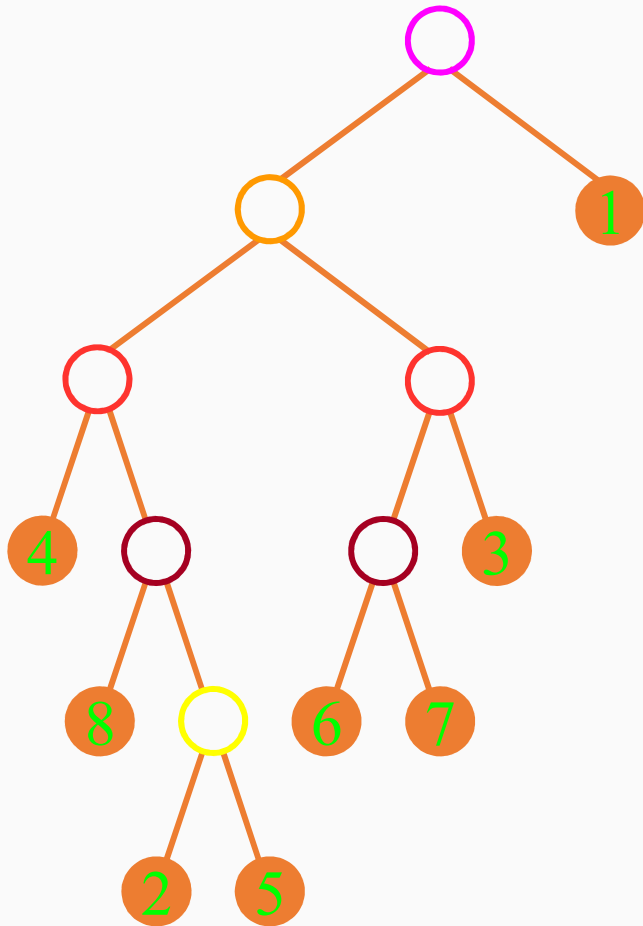
Hüllkörperhierarchien (5)

- Greedy Algorithmus zur Erzeugung „guter“ kD-Baum Hierarchien in $O(n \log n)$ Zeit
 - Starte mit allen Objekten in einem Hüllkörper
 - Suche „beste“ Unterteilung in jedem Schritt
 - Abschätzung für Gesamtoberfläche $A(T)$ aller Hüllkörper einer Teilhierarchie T :

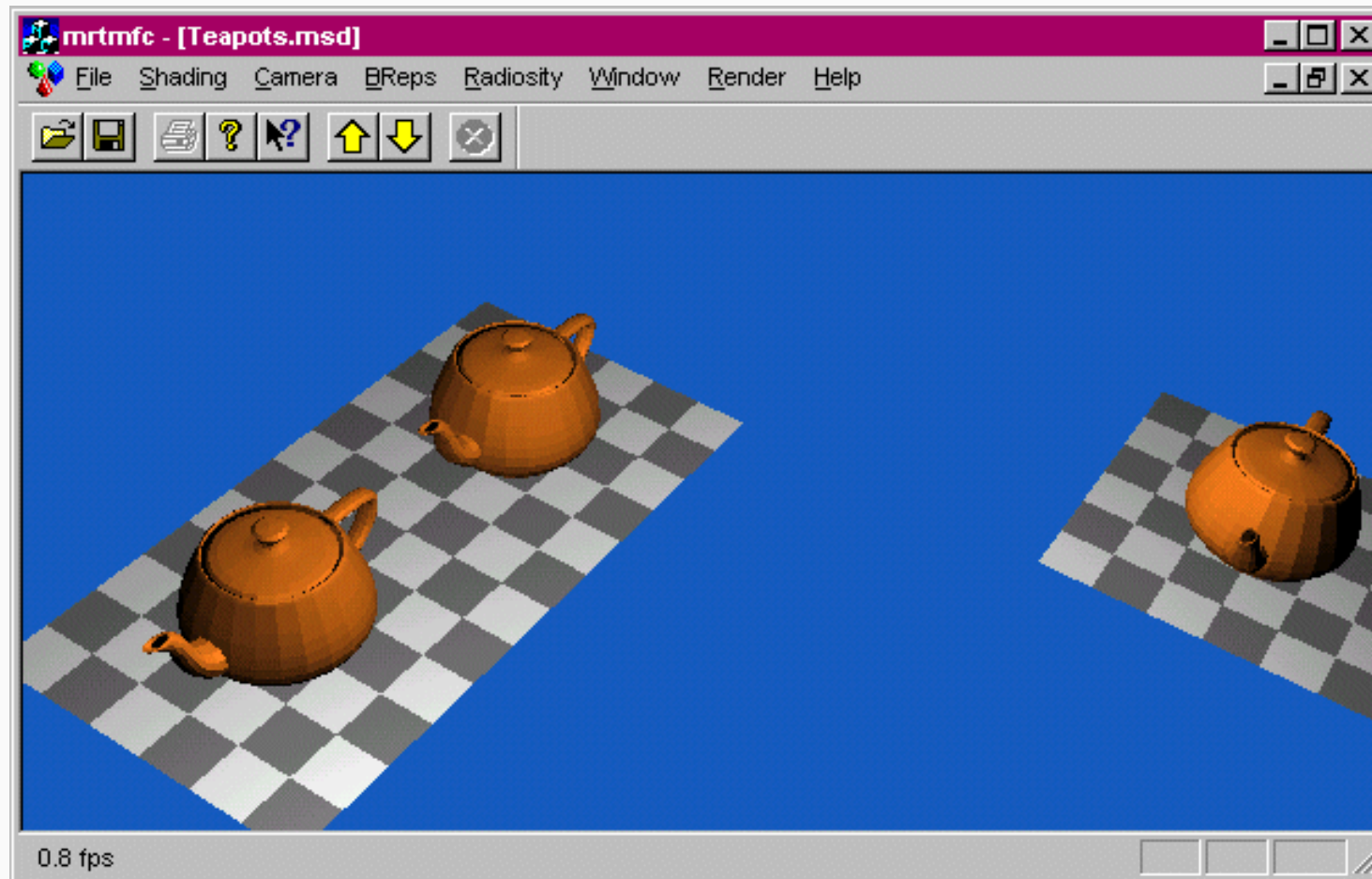
$$A(T) \approx \log n_T \cdot A_H(T)$$

- n_T : Anzahl der Elemente von T
 - $A_H(T)$: Oberfläche des Hüllkörpers von T
- Berechne $A(T)$ für alle möglichen Unterteilungen
- Aufwand pro Schritt $O(n)$

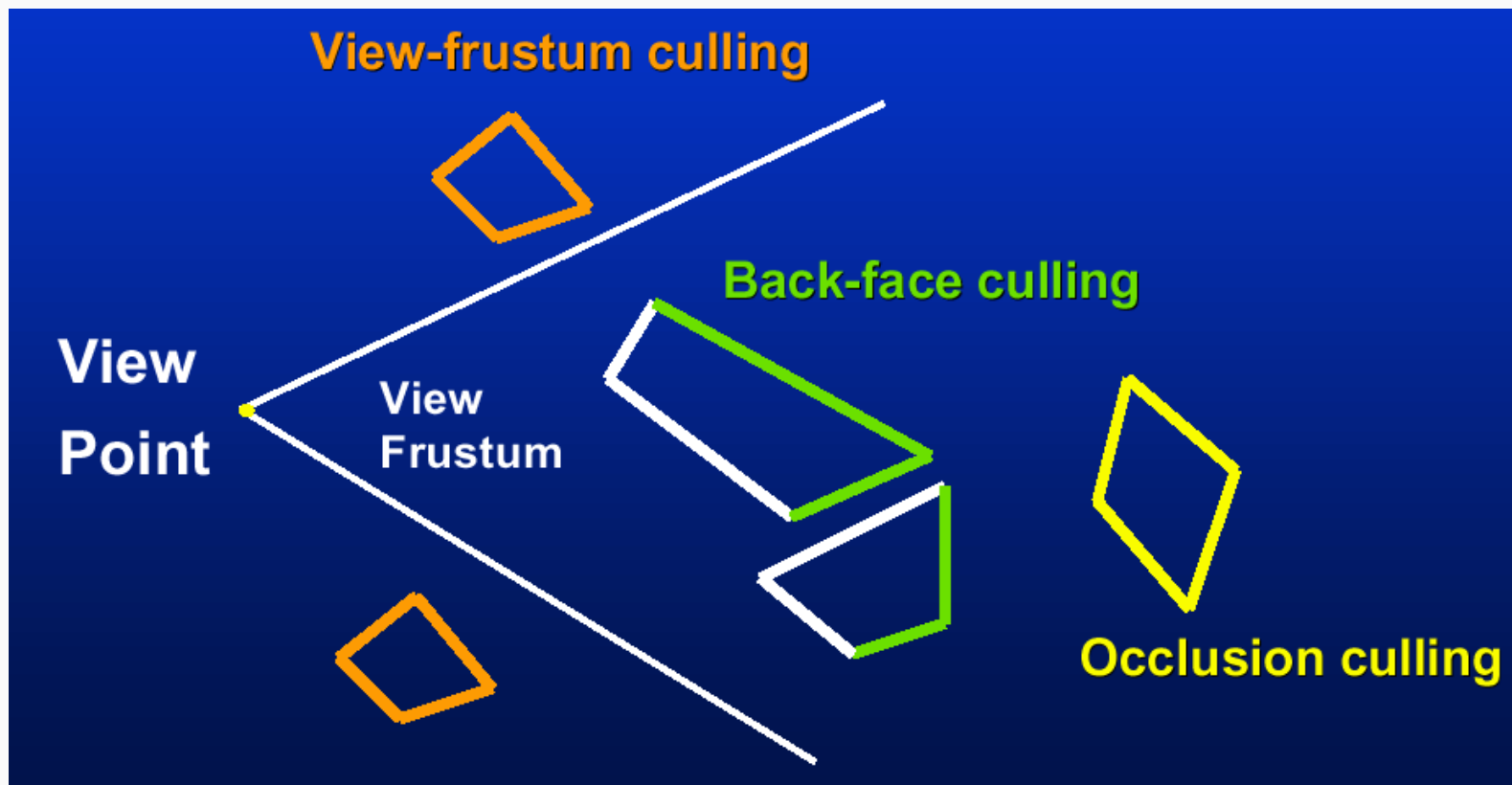
Hüllkörperhierarchien (6)



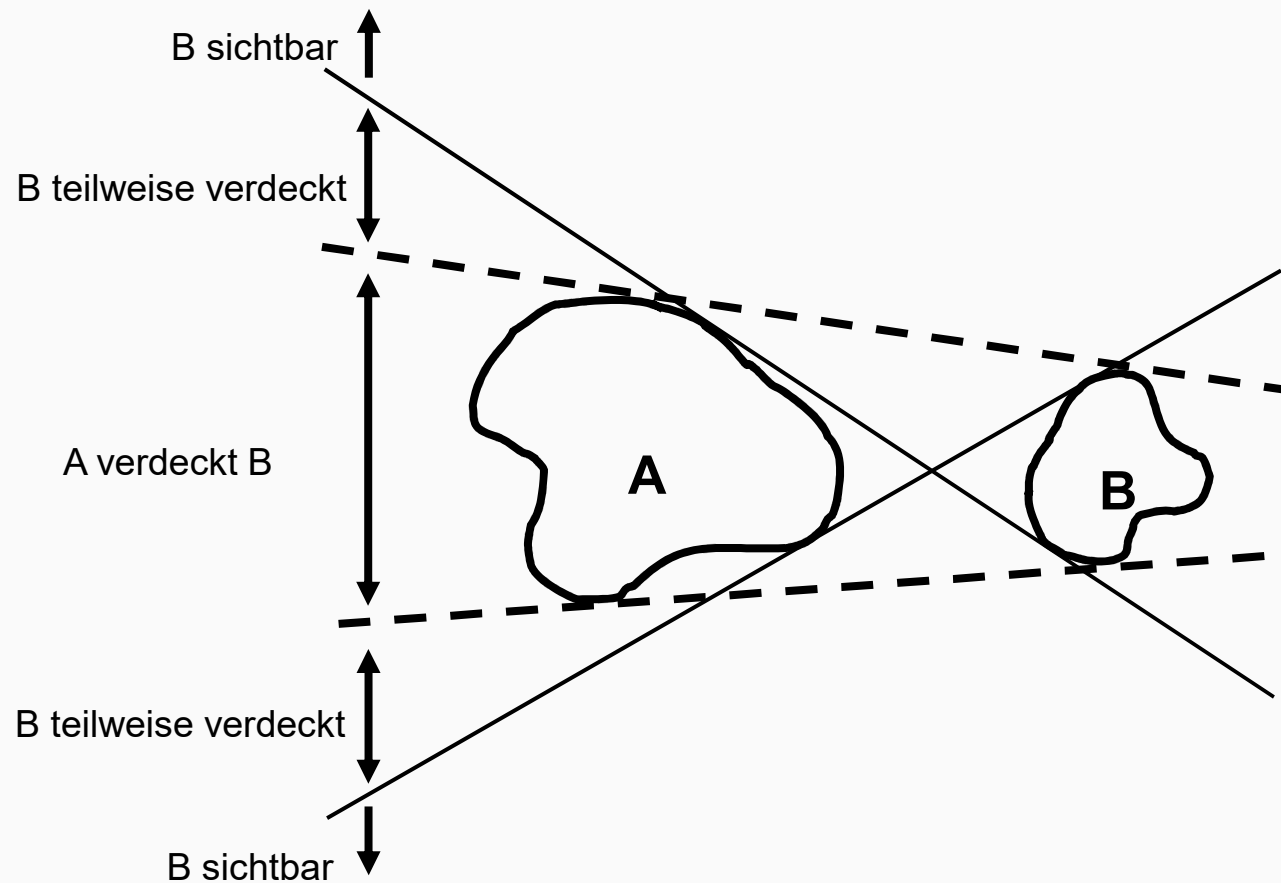
Hüllkörperhierarchien (7)



Occlusion Culling (1)

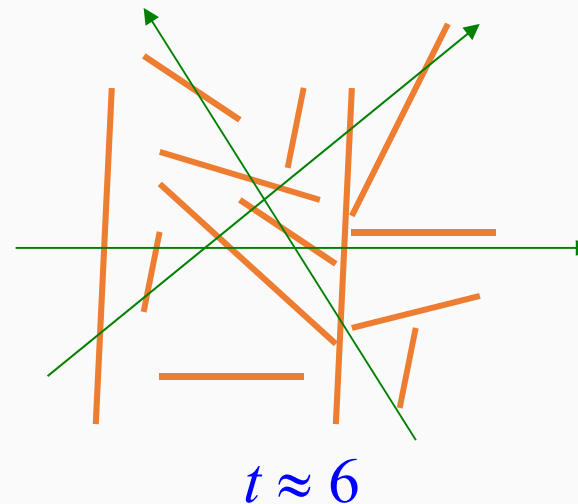
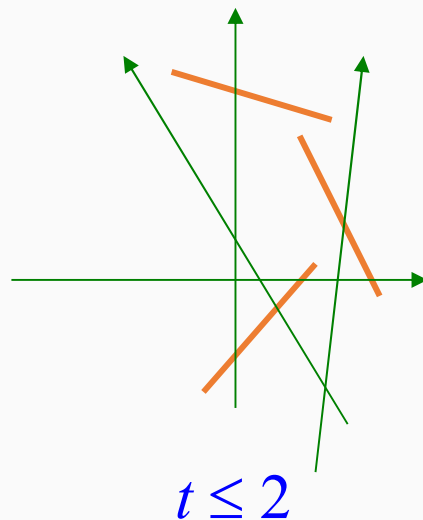


Occlusion Culling (2)



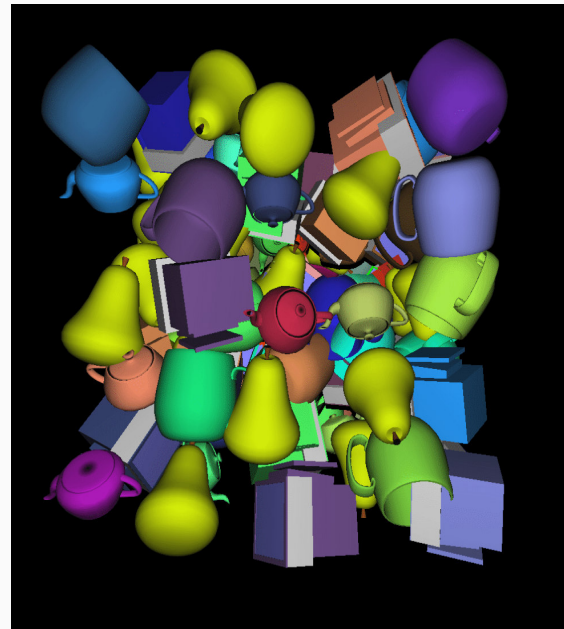
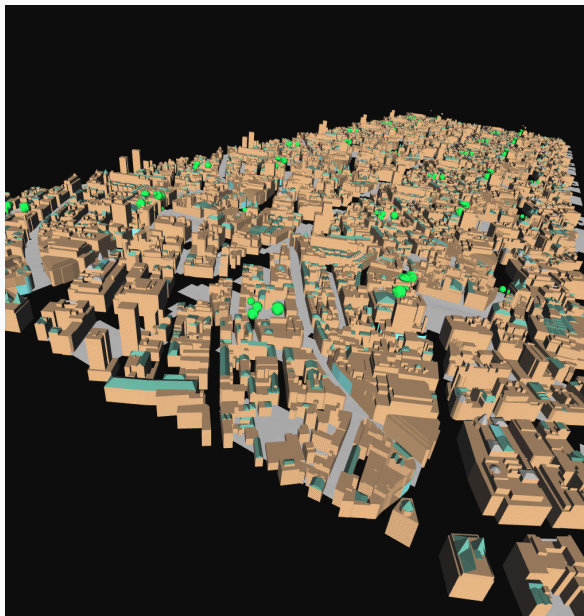
Occlusion Culling (3)

- Immer dann interessant, falls Objekte durch andere Objekte verdeckt werden
- Verdeckungstiefe t (beobachtungs- und richtungsabhängig)
 - Anzahl Schnittpunkte eines Strahls durch die Szene



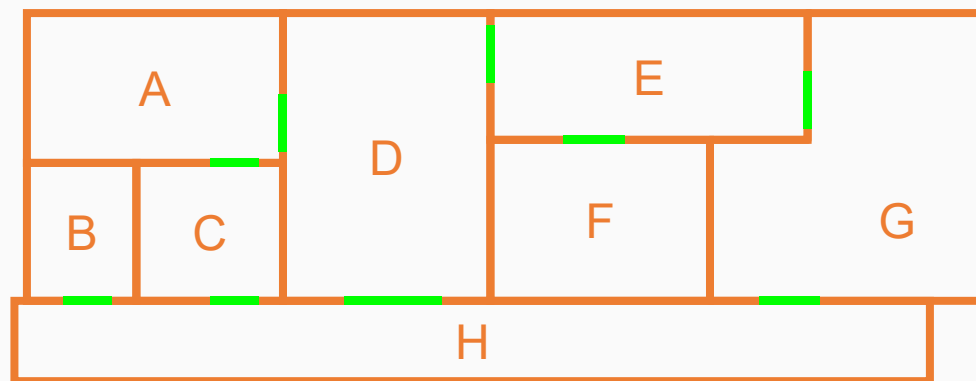
Occlusion Culling (4)

- Beispiele hoher Verdeckungstiefe



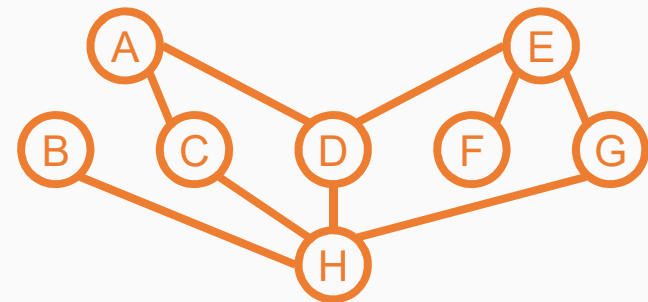
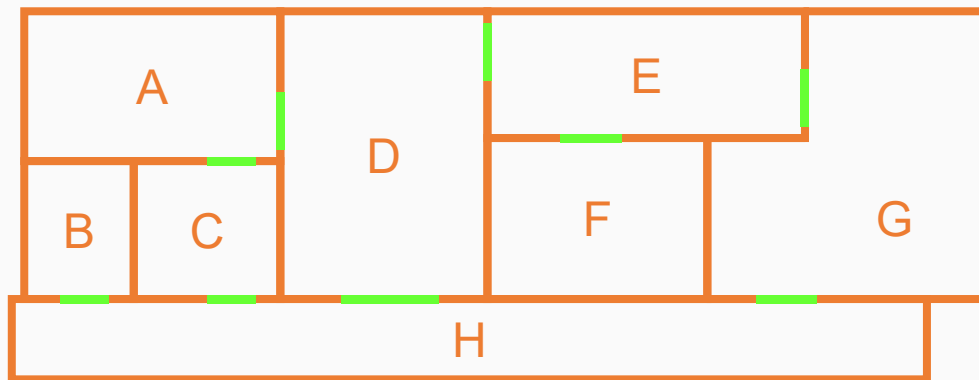
Zellen und Portale (1)

- Ziel: Durchwandern von Gebäuden und Städten
- Durchsichtige Portale verbinden die Zellen
 - Türen, Fenster, Öffnungen, ...
- Beobachtung: Zellen sehen einander nur durch die Portale



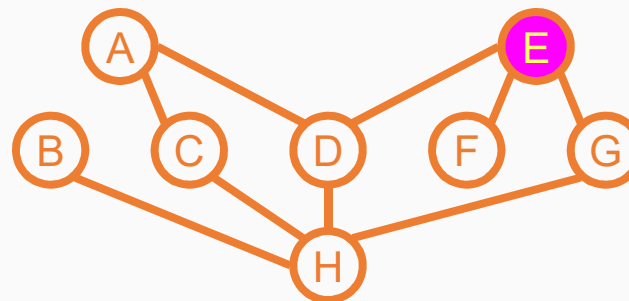
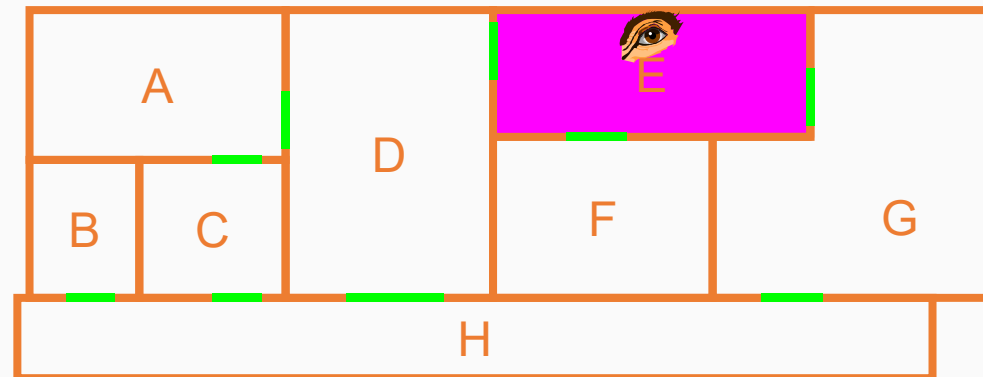
Zellen und Portale (2)

- Idee:
 - Berechne Nachbarschaftsgraphen zwischen Zellen
 - Beginne mit Zelle, die Augpunkt enthält und traversiere Graphen, wobei sichtbare Zellen gerendert werden
 - Eine Zelle ist nur dann sichtbar, wenn sie durch eine Folge von Portalen gesehen werden kann
 - Zellvisibilität reduziert sich auf das Testen von Portalfolgen



Zellen und Portale (3)

- Beispiel:

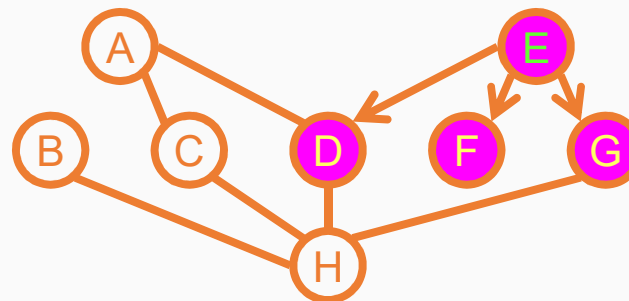
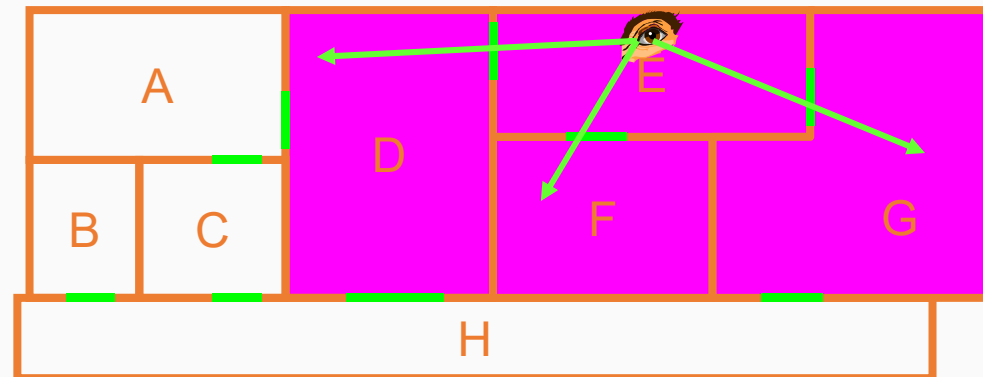


Portalfolgen:

E

Zellen und Portale (4)

- Beispiel:



Portalfolgen:

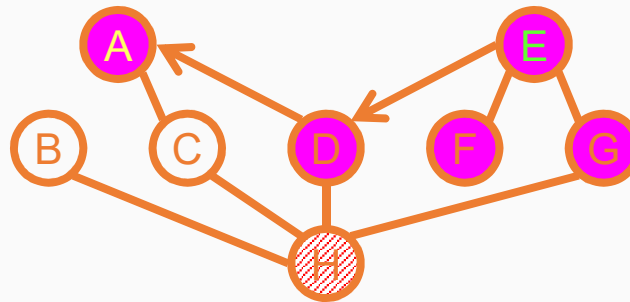
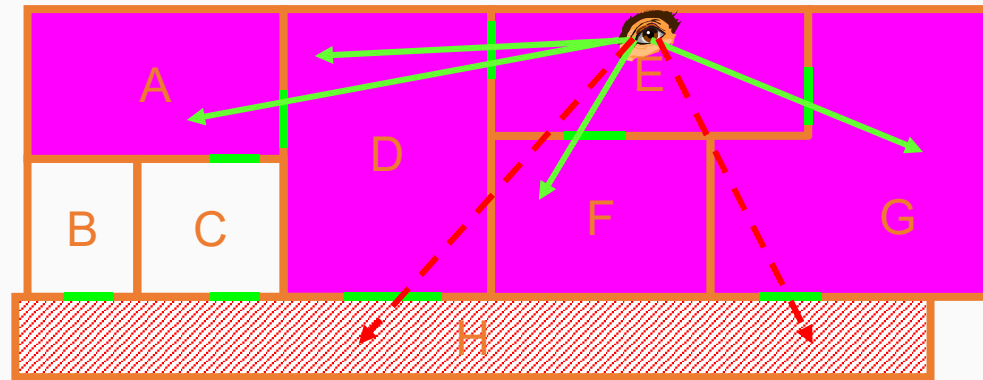
$E \rightarrow D$

$E \rightarrow F$

$E \rightarrow G$

Zellen und Portale (5)

- Beispiel:



Portalfolgen:

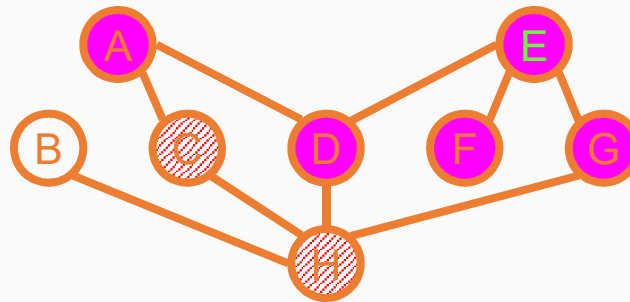
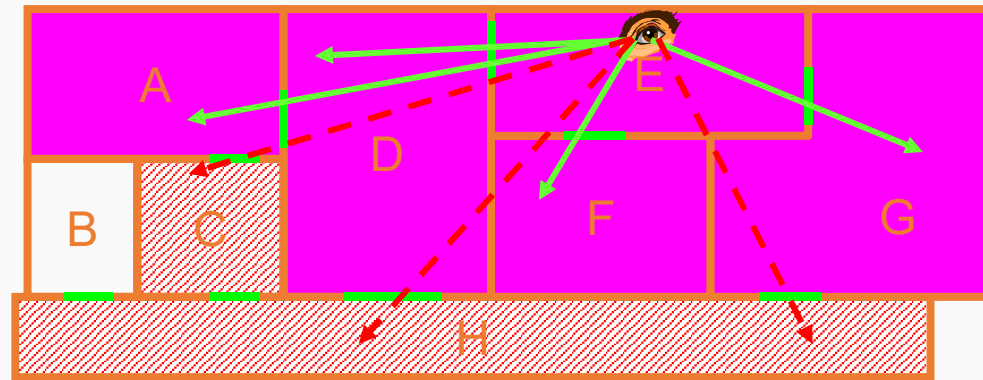
$E \rightarrow D \rightarrow A$

~~$E \rightarrow D \rightarrow H$~~

~~$E \rightarrow G \rightarrow H$~~

Zellen und Portale (6)

- Beispiel:

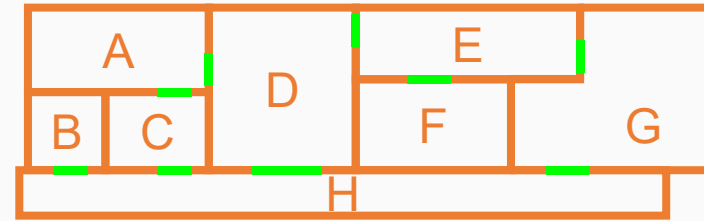


Portalfolgen:

 ~~$E \rightarrow D \rightarrow A \rightarrow C$~~

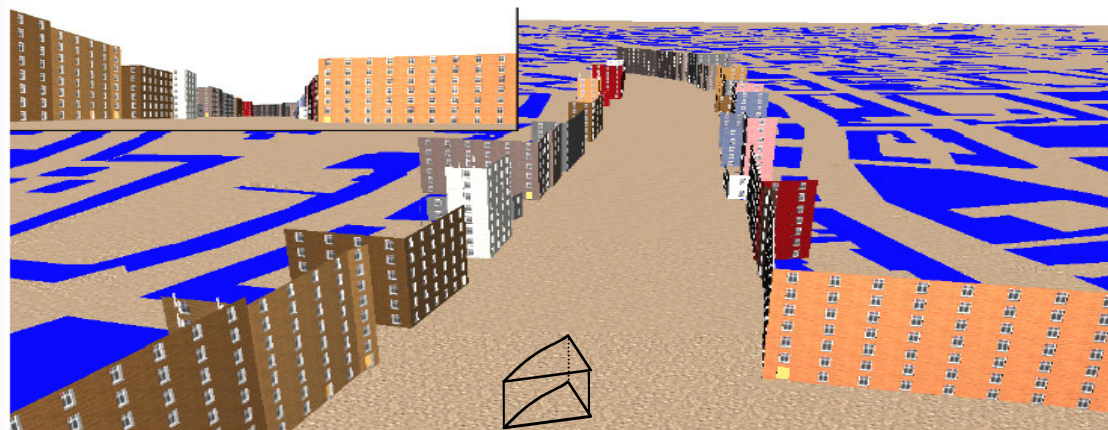
Zellen und Portale (7)

- Weitere Vorverarbeitung:
 - Bestimme welche Zellen eine bestimmte Zelle nie sehen können
 - z.B. H sieht nie F, B sieht nur H
 - Speichere zu jeder Zelle alle Zellen, die möglicherweise eingesehen werden können
 - z.B. A sieht C, H, D, E



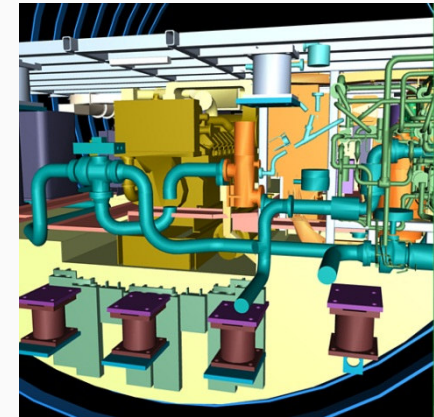
Zellen und Portale (8)

- Algorithmen zur Vorverarbeitung:
 - S. Teller, *Visibility Computations in Densely Occluded Polyhedral Environments*. PhD thesis, 1992
 - D. Luebke, C Georges. *Portals and Mirrors: Simple, Fast Evaluation of Potentially Visible Sets*. I3D, 1995
 - F. Durand. *3D Visibility. Analytical Study and Applications*. PhD thesis, 1999



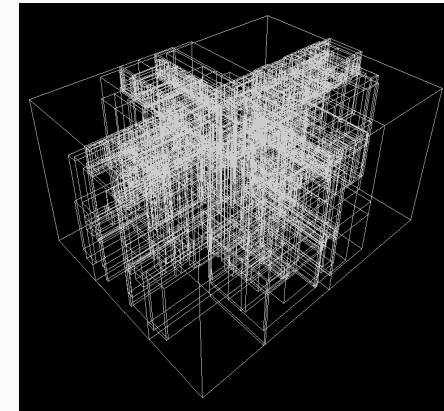
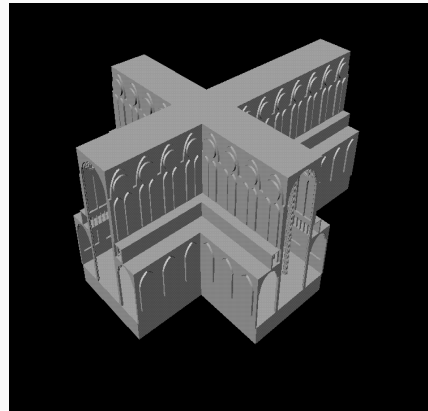
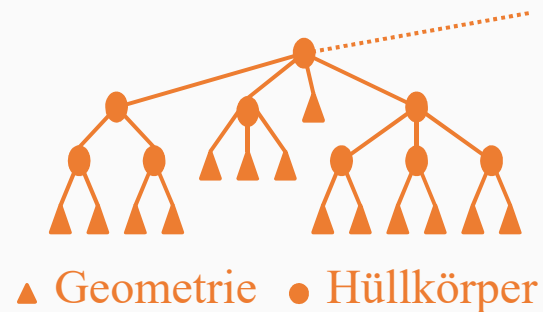
Zellen und Portale (9)

- Anwendungsgebiete
 - Computerspiele
 - Gebäude
 - Städte
- Nicht geeignet für CAD-Daten
 - Flugzeuge
 - Schiffe
 - Industrieanlagen
- Nicht geeignet für natürliche Objekte
 - Pflanzen
 - Wälder



Hierarchisches Occlusion Culling (1)

- Vorverarbeitung:
 - Hüllkörperhierarchie generieren



- Rendering:
 - Verarbeite BB von vorne nach hinten:
 - Teste ob BB sichtbar ist oder nicht
 - Falls BB verdeckt, werden Kinder nicht weiterbetrachtet, sonst werden sie weiterverarbeitet

Hierarchisches Occlusion Culling (2)

- Grundidee:
 - Ausnutzen der View-Frustum Culling Hierarchie auch für Occlusion Culling
 - Änderung der Traversierungsreihenfolge:
 - Für View-Frustum Culling irrelevant
 - Für Occlusion Culling: vorderer Kindknoten vor dem hinterem
 - Prioritätswarteschlange statt Traversierung mit Stack

Hierarchisches Occlusion Culling (3)

- Algorithmus:

```
Queue.insert(Root)
```

```
while (!Queue.empty)
```

```
    Head = Queue.getHead()
```

```
    Queue.remove(Head)
```

```
    if (!Head.viewFrustumCulled)
```

```
        if (!Head.occluded)
```

```
            if (Head.isLeaf)
```

```
                Head.render
```

```
            else
```

```
                Queue.insert(Head.Children)
```

Hierarchisches Occlusion Culling (4)

- Hardware-Unterstützung
 - Occlusion Culling Flag
 - HP-Visualize fx graphics hardware
 - Ab GeForce3
 - Alle Radeon
 - Scankonvertierung von Polygonen, ohne in Framebuffer zu schreiben; Flag, ob Pixel des Polygons den z-Test passiert haben
 - HP Occlusion-Test (True / False)
 - ARB Occlusion-Query (# Pixel)
- http://developer.nvidia.com/object/gdc_occlusion.html

Hierarchisches Occlusion Culling (5)

- HP Occlusion Test

1. Schreiben in z-Buffer und Bildspeicher abschalten (optional)

```
glDepthMask(GL_FALSE)
```

```
glColorMask(GL_FALSE, GL_FALSE, GL_FALSE, GL_FALSE)
```

2. Occlusion Test einschalten

```
glEnable(GL_OCCLUSION_TEST_HP)
```

3. (Bounding) Geometrie rendern

4. Occlusion Test abschalten

```
glDisable(GL_OCCLUSION_TEST_HP)
```

5. Occlusion Test Ergebnis abfragen

```
glGetBooleanv(GL_OCCLUSION_TEST_RESULT_HP, &result)
```

Hierarchisches Occlusion Culling (6)

- ARB Occlusion-Query

1. Queries erzeugen

```
glGenQueriesARB(num, queries)
```

2. Schreiben in z-Buffer und Bildspeicher abschalten (optional)

```
glDepthMask(GL_FALSE)
```

```
glColorMask(GL_FALSE, GL_FALSE, GL_FALSE, GL_FALSE)
```

3. Occlusion Test einschalten

```
glBeginQueryARB(GL_SAMPLES_PASSED_ARB, query[i])
```

4. (Bounding) Geometrie rendern

5. Occlusion Test abschalten

```
glEndQueryARB(GL_SAMPLES_PASSED_ARB)
```

Hierarchisches Occlusion Culling (7)

- ARB Occlusion-Query (Forts.)

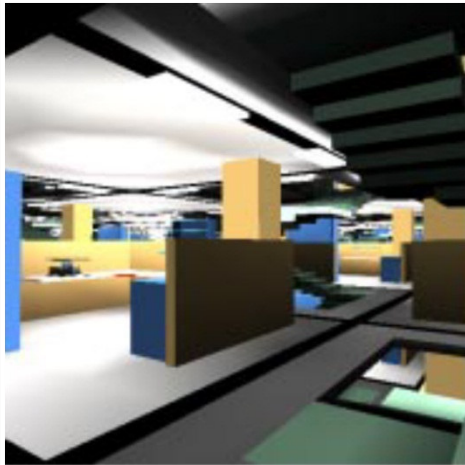
6. Abfragen, ob Query Ergebnis verfügbar ist

```
glGetQueryObjectivARB(query[i],  
    GL_QUERY_RESULT_AVAILABLE_ARB, &result)
```

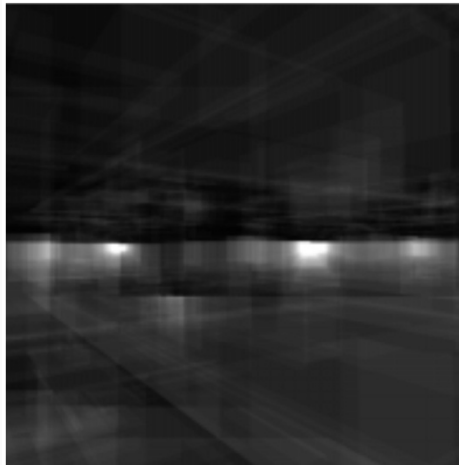
7. Occlusion Query Ergebnis abfragen

```
glGetQueryObjectivARB(query[i],  
    GL_QUERY_RESULT_ARB, &result)
```

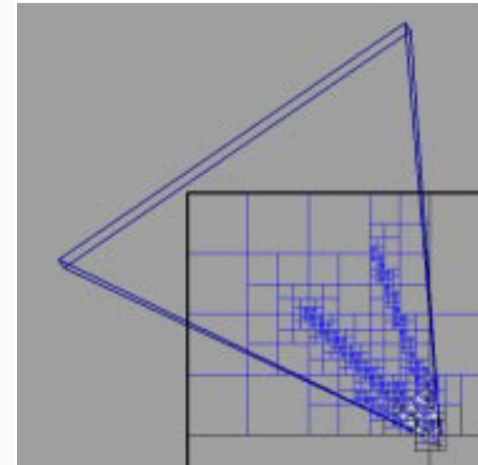
Hierarchisches Occlusion Culling (8)



538.000.000 Polygone



Farbcodierte Tiefenkomplexität



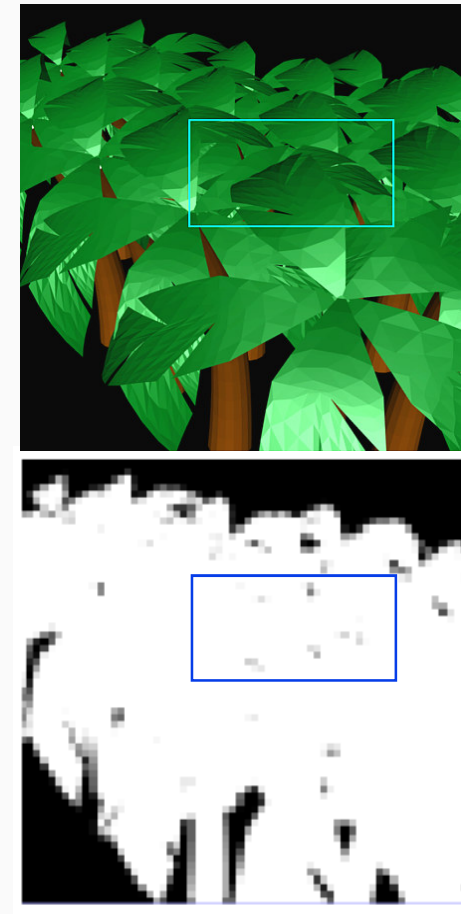
Nicht verdeckte Octree Zellen

Aggressives approximatives Culling (1)

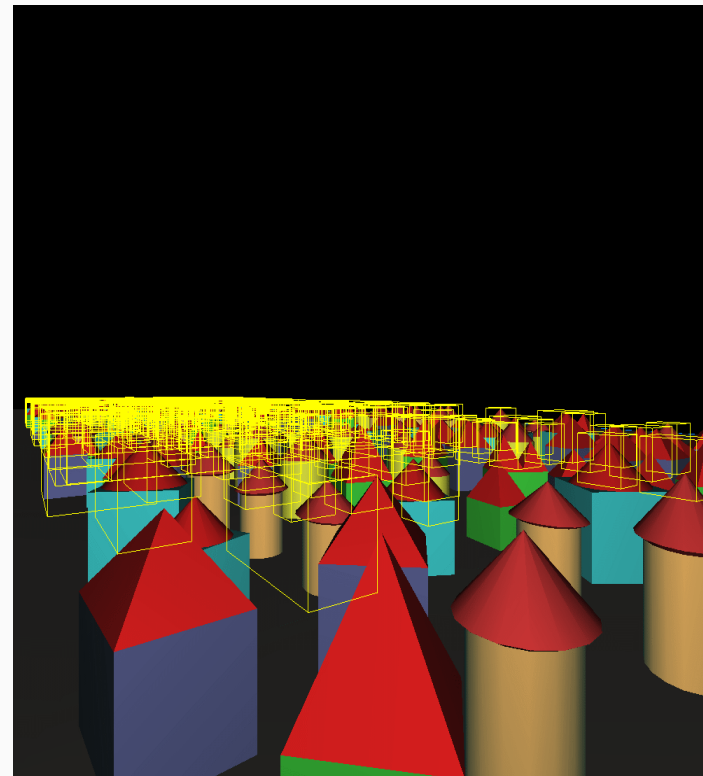
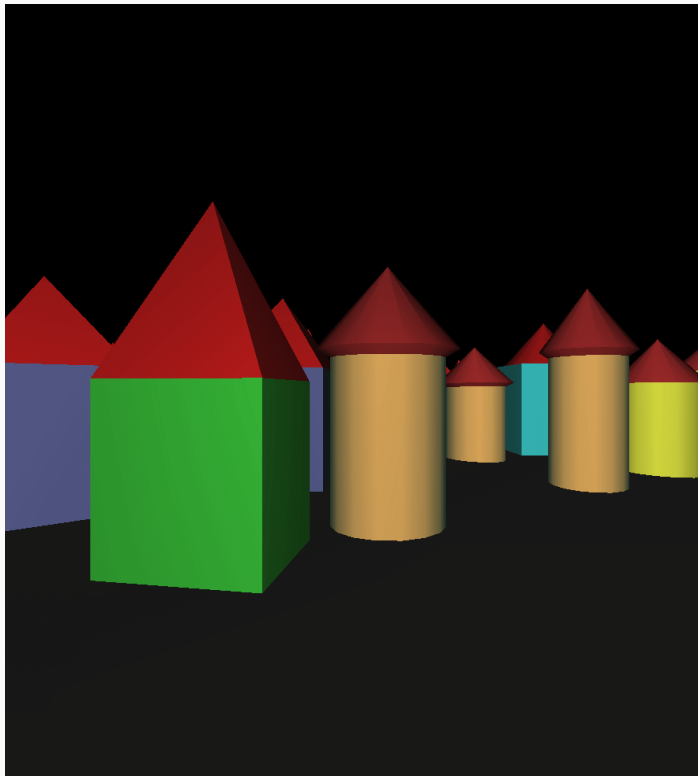
- Pessimistischer (konservativer) Ansatz:
 - Wenn auch nur ein Pixel des Hüllkörpers sichtbar ist, kann auch ein Pixel des Objektes sichtbar sein
 - Objekt zeichnen, wenn Query Ergebnis $\neq 0$
 - Nachteil:
 - Oft sind äußere Teile der Hüllkörper sichtbar, an denen sich keine Objektpixel befinden

Aggressives approximatives Culling (2)

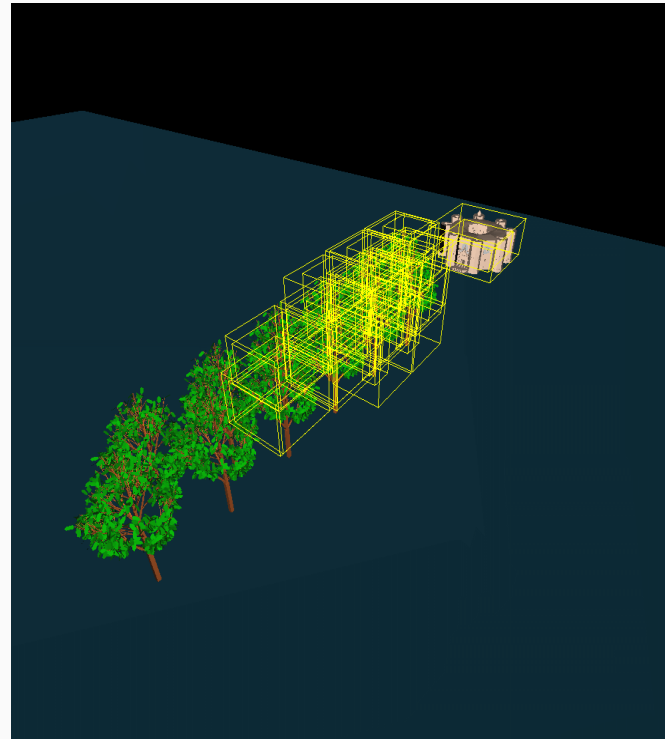
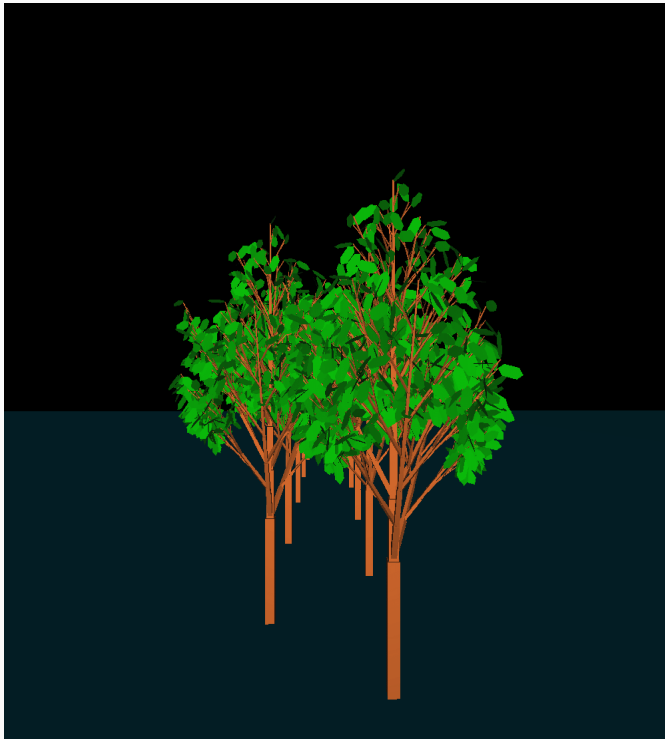
- Ignoriere kaum sichtbare Objekte:
 - Sichtbarkeitswahrscheinlichkeit des Objektes gering, wenn nur wenige Pixel des Hüllkörpers sichtbar
 - Objekt nur zeichnen, wenn Query Ergebnis $\geq n$
 - Eventuell „kleine“ Löcher in einem oder zwischen Objekten
 - Entspricht Tiefpaßfilter, der Rauschen entfernt



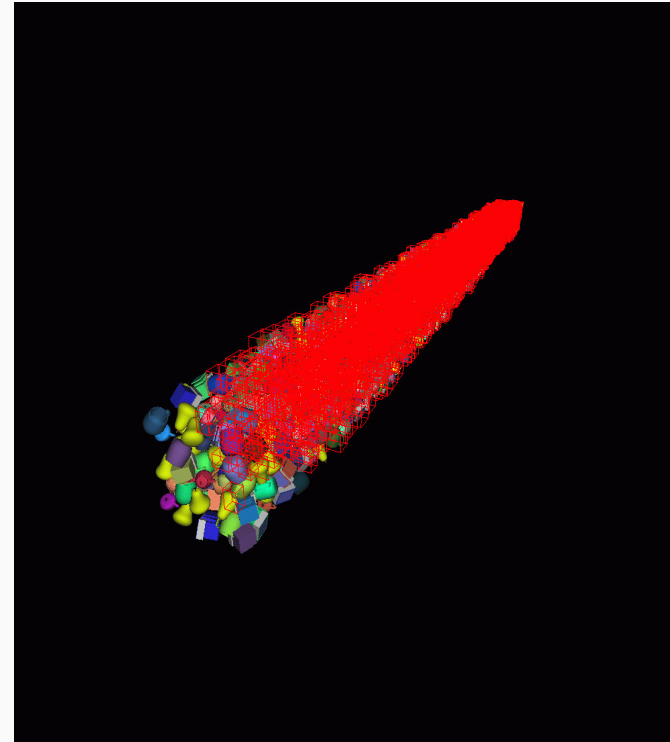
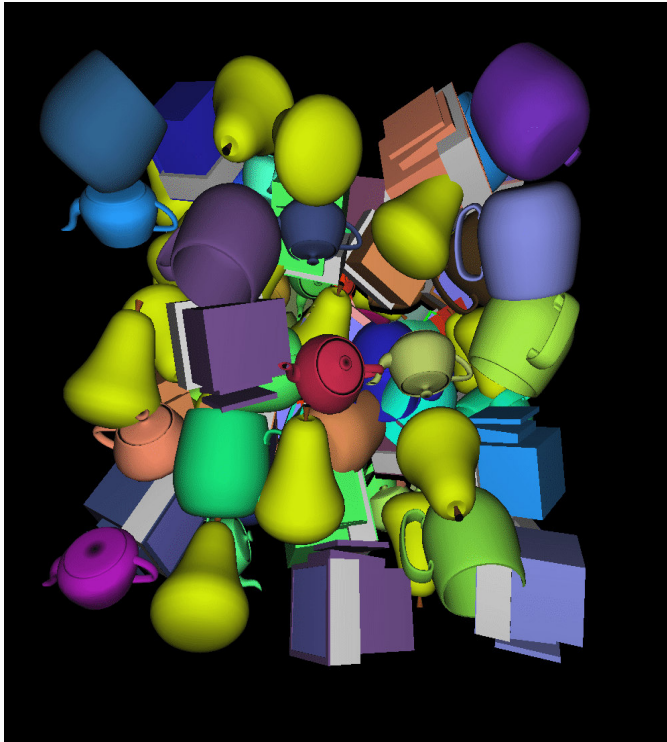
Aggressives approximatives Culling (3)



Aggressives approximatives Culling (4)



Aggressives approximatives Culling (5)



Kohärentes Hierarchisches Occlusion Culling (CHC) (1)

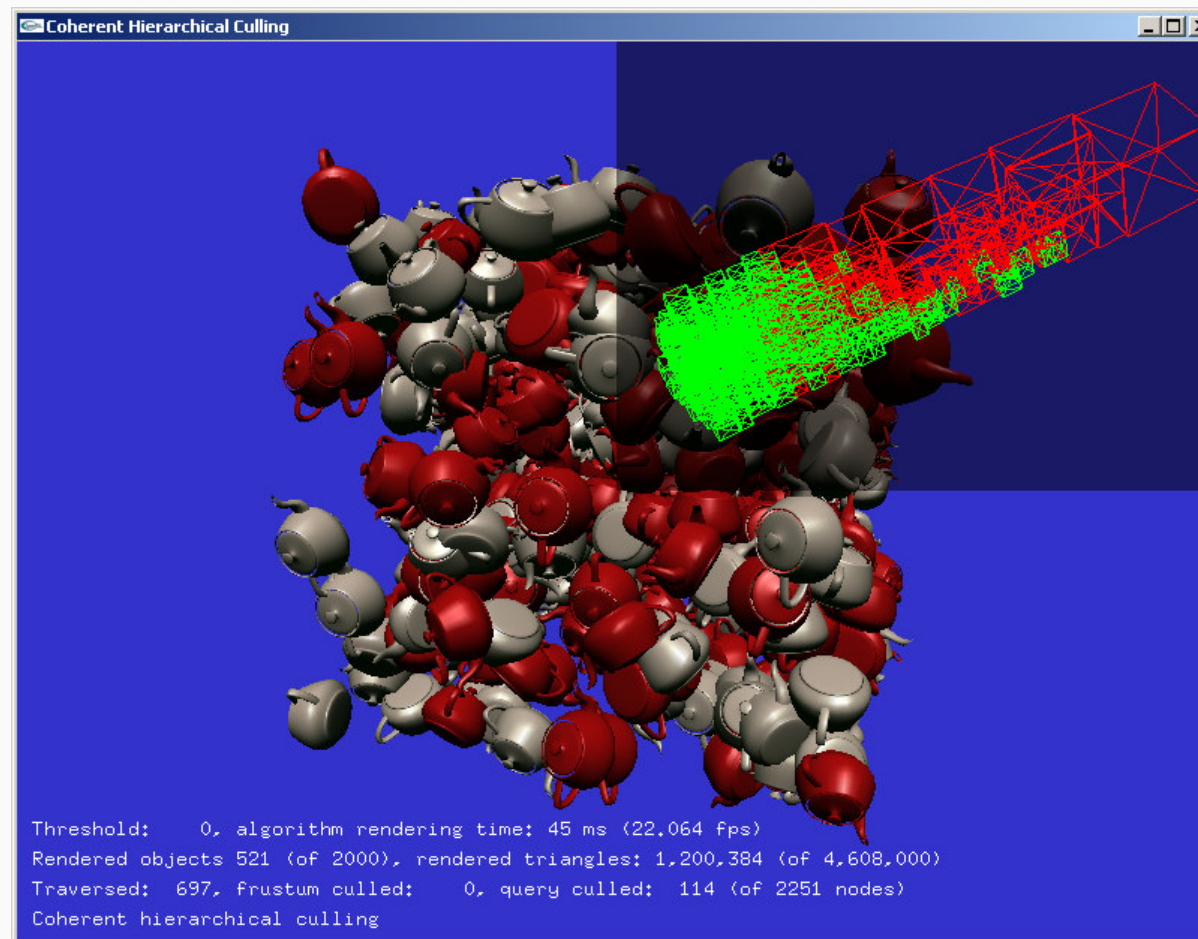
- Problem:
 - Große Wartezeit zwischen Test und Ergebnis (sehr lange Pipelines auf Graphikkarten)
- Lösungsmöglichkeiten:
 - Unnötige Tests vermeiden
 - ARB Occlusion-Query erlaubt Ausführung mehrerer paralleler Tests
 - Gleichzeitiger Test nicht-überlappender Knoten
 - Nicht auf Ergebnis warten, wenn Objekt sichtbar war (zeitliche Kohärenz)

Kohärentes Hierarchisches Occlusion Culling (CHC) (2)

```
Queue.insert(Root)
while (!Queue.empty && !TestList.empty)
    if (!Queue.empty)
        Head = Queue.Head
        Queue.remove(Head)
        if (!Head.viewFrustumCulled)
            if (Head.isLeaf || !Head.WasVisible)
                Head.issueQuery
            if (Head.WasVisible)
                if (Head.isLeaf)
                    Head.render
                else
                    Queue.insert(Head.Children)
        Head.Visible = false
```

```
while (!TestList.empty &&
        TestList.Head.testFinished)
    Head = TestList.Head
    TestList.remove(Head)
    if (!Head.occluded)
        if (!Head.WasVisible)
            if (Head.isLeaf)
                Head.render
            else
                Queue.insert(Head.Children)
    Head.Visible = true
    Head.setParentsVisible
```

Kohärentes Hierarchisches Occlusion Culling (CHC) (3)



- J. Bittner, P. Wonka. *Visibility in computer graphics*. Environment and Planning B: Planning and Design 30, 5, pp. 729-756, 2003
- J. Bittner, M. Wimmer, H. Piringer, W. Purgathofer. *Coherent hierarchical culling: hardware occlusion queries made useful*. Computer Graphics Forum (Eurographics) 23, 2, pp. 615-624, 2004
- M. Guthe, Á. Balázs, R. Klein. *Near optimal hierarchical culling: performance driven use of hardware occlusion queries*. Rendering Techniques (Eurographics Symposium on Rendering), pp. 207-214, 2006
- O. Mattausch, J. Bittner, M. Wimmer. *CHC++: Coherent Hierarchical Culling Revisited*. Computer Graphics Forum (Eurographics), 27, 2, pp. 221-230, 2008